

Dynamic Research Paper Retrieval System Using Retrieval Augmented Generation

Dissertation-II

Submitted in partial fulfillment of the requirements for the degree of

**Master of Computer Applications
in
Department of Computer Applications**

by

SAHANA BHAT U

23MCA0128

**Under the guidance of
Dr. KISHORERAJA P C**

School of Computer Science Engineering and Information Systems

VIT, Vellore



VIT[®]
Vellore Institute of Technology
(Deemed to be University under section 3 of UGC Act, 1956)

April, 2025

Dynamic Research Paper Retrieval System Using Retrieval Augmented Generation

Dissertation-II

Submitted in partial fulfillment of the requirements for the degree of

**Master of Computer Applications
in
Department of Computer Applications**

by

SAHANA BHAT U

23MCA0128

**Under the guidance of
Dr. KISHORERAJA P C**

**School of Computer Science Engineering and Information Systems
VIT, Vellore**



April, 2025

DECLARATION

I hereby declare that the **PMCA699J - Dissertation-II** entitled “Dynamic Research Paper retrieval using Retrieval Augmented Generation” submitted by me, for the award of the degree of Master of Computer Applications in Department of Computer Applications, School of Computer Science Engineering and Information Systems to VIT is a record of bonafide work carried out by me under the supervision of Dr. Kishoreraja P C, Professor Grade 1, SCORE, VIT, Vellore.

I further declare that the work reported in this dissertation has not been submitted and will not be submitted, either in part or in full, for the award of any other degree ordiploma in this institute or any other institute or university.

Place: Vellore

Date:

Signature of the Candidate

CERTIFICATE

This is to certify that the **PMCA699J -Dissertation-II** entitled “Dynamic Research Paper retrieval using Retrieval Augmented Generation” submitted by Sahana Bhat U 23MCA0128, SCORE, VIT, for the award of the degree of Master of Computer Applications in Department of Computer Applications, is a record of bonafide work carried out by him / her under my supervision during the period, 13. 12. 2024 to 17.04.2025, as per the VIT code of academic and research ethics.

The contents of this report have not been submitted and will not be submitted either in part or in full, for the award of any other degree or diploma in this institute or any other institute or university. The dissertation fulfill the requirements and regulations of the University and in my opinion meets the necessary standards for submission.

Place: Vellore

Date:

Signature of the VIT-SCORE - Guide

Internal Examiner

External Examiner

Head of the Department
Department of Computer Applications

ACKNOWLEDGEMENTS

It is my pleasure to express with a deep sense of gratitude to my Dissertation- II guide **Dr. Kishoreraja P C**, Professor Grade 1, School of Computer Science Engineering and Information Systems, Vellore Institute of Technology, Vellore for his constant guidance, continual encouragement, in my endeavor. My association with him is not confined to academics only, but it is a great opportunity on my part to work with an intellectual and an expert in the field of Artificial Intelligence.

"I would like to express my heartfelt gratitude to Honorable Chancellor **Dr. G Viswanathan**; respected Vice Presidents **Mr. Sankar Viswanathan**, **Dr. Sekar Viswanathan**, Vice Chancellor **Dr. V. S. Kanchana Bhaaskaran**; Pro-Vice Chancellor **Dr. Partha Sharathi Mallick**; and Registrar **Dr. Jayabarathi T.**

My whole-hearted thanks to Dean **Dr. Daphne Lopez**, School of Computer Science Engineering and Information Systems, Head, Department of Computer Applications **Dr. E Vijayan**, MCA Project Coordinator **Dr. Karthikeyan P**, SCORE School Project Coordinator **Dr. Thandeeswaran R**, all faculty, staff and members working as limbs of our university for their continuous guidance throughout my course of study in unlimited ways.

It is indeed a pleasure to thank my parents and friends who persuaded and encouraged me to take up and complete my dissertation successfully. Last, but not least, I express my gratitude and appreciation to all those who have helped me directly or indirectly towards the successful completion of the dissertation.

Place: Vellore

Date:

Student Name

Executive Summary

The traditional method of paper management, often found in static repositories that cannot provide efficient retrieval of any information required by researchers, is intended to be changed by the implementation of web scraping in research papers to acquire data from diverse sources online. The gathered metadata will comprise titles, authors, abstracts, keywords, and even the details about the publications involved, all of which are then processed and stored within a graph database. This method of storage allows for a rich, interconnected representation of research topics and relationships between them, thereby fostering an in-depth understanding of the knowledge landscape. The system uses Retrieval-Augmented Generation methodology to enhance usability, which integrates the graph database with a language model for advanced query handling and response generation. This way, the system can dynamically retrieve contextually relevant papers based on user queries, ensuring precision and relevance. With the semantic intent behind user inputs understood, the language model delivers accurate and insightful results, tailored to the specific research needs of users.

This is the innovative combination of web scraping, graph databases, and RAG methodology transforming the way researchers access and use academic knowledge. It will streamline the exploration of cross-disciplinary insights, improve accessibility to a diverse range of research, and support efficient knowledge discovery for academia and beyond.

Keywords: RAG, Graph database, WebScraping, Generative AI, Research Papers.

CONTENTS	Page No.
Acknowledgement	v
Executive Summary	vi
Table of Contents	vii
List of Figures	ix
1.Introduction	1
1.1 General Introduction	1
1.2 Problem Statement	2
1.3 What is Hybrid RAG?	3
1.4 How hybrid rag works?	4
1.5 Why Hybrid RAG?	6
1.6 Objectives	8
1.7 Gap Identification	10
1.8 Scope of the Project	11
2.Literature Survey	13
2.1 Literature Survey	13
2.2 Findings of the Literature survey	20
3 Dissertation Goal and Discription	23
3.1 Existing System	23
3.2 Disadvantages of Existing System	23
3.3 Proposed System	26
3.4 Advantages of Proposed System	28
3.5 Use-case Scenario	29
4 DESIGN APPROACH AND DETAILS	31
4.1 Detail design Approach	31

4.2 System Architecture	33
4.3 UML Diagrams	34
4.4 Constraints of the Dissertation	36
4.5 Methodology	37
5 Technical Specifications	42
5.1 Dataset Collection	42
5.2 Neo4j graph database	42
5.3 Vector Embedding	43
5.4 Weaviate vector database	44
5.5 Summary and Objective Generation	44
5.6 Output of the system	45
6 Hardware and Software Requirements	46
6.1 Hardware Requirements	46
6.2 Software Requirements	46
7 Dissertation Demonstration	48
7.1 Code to extract data and save in pdf	48
7.2 Code to save the data in graph database	48
7.3 Code to save the data in vector database	49
7.4 Code to retrieve the data using hybrid RAG	50
7.5 Output of the system	54
8 Conclusion	58
8.1 Conclusion	58
8.2 Future Scope	58
9 References	60

List of Figures

Figure No.	Title	Page No.
1.1	Flow Diagram of RAG	3
1.2	Hybrid Search Flow	6
4.1	System Architecture	34
4.2	Class Diagram	35
4.3	Use case Diagram	36
5.1	Neo4j Instance	43
7.1	Virtual Environment Creation	55
7.2	Running Streamlit app	55
7.3	Keyword Search-1	56
7.4	Keyword Search-2	56
7.3	Keyword Search-3	57

CHAPTER 1

DISSERTATION INTRODUCTION

1.1 General Introduction

A literature survey, or literature review, is an important part of research, it provides a strong foundation on which new research element is built. By reviewing and analyzing existing studies, researchers gain a deep understanding of the knowledge in their field, including existing key theories, methodologies, and findings. This process allows them to understand the previous work, gain knowledge on current trends, and identify the gaps or unexplored areas in the research. Conducting a proper literature survey helps to get clarity on the originality of the work and helps in getting a new perspective of the existing problem. Moreover, it allows researchers to learn from previous research methods and results, which can improve their knowledge in their work. Without a literature survey, new research might repeat existing work, adding little value to the field and wasting time and resources.

Traditionally, a literature survey is conducted in a structured process that involves identifying the related papers, reading and analyzing the research articles. Researchers usually start by defining keywords related to their topic, which they use to search in databases like Google Scholar, PubMed, IEEE Xplore and other repositories. After generating a list of relevant papers, the researcher must carefully analyse each paper, extracting important information such as research objectives, methodologies, results, and conclusions. This information is then organized, and stored in a structured format which will help the researchers to get a clear picture between each papers. Finally, once the insights of the paper gathered they are converted into a narrative that highlights existing knowledge, methodological approaches, and research gaps. While essential, this process can be very much time-consuming and requires researchers to track and organize a large volume of information.

With the exponential growth of published research, manually conducting literature surveys have become very much challenging. There are thousands of new papers are published every day, making it difficult for researchers to keep up track with latest developments in their fields. This is where automation can offer significant benefits. Advances in artificial intelligence (AI) and natural language processing (NLP)

have paved the way for tools that can ease the literature review process. Automated literature survey systems can easily search and retrieve relevant papers based on keywords, going beyond simple keyword matching to understand the context of the user's query.

Additionally, these systems can generate summaries of research papers, highlighting key findings and helping researchers understand the content without having to read each paper in full. With AI-driven tools, researchers can organize large volumes of data, filter results by relevance, and quickly identify research trends, allowing them to focus on analysis and interpretation rather than data gathering. By automating the more repetitive and time-consuming aspects of literature surveys, these systems free up researchers' time and enable a more efficient approach to keeping up with current research.

1.2 Problem Statement

The situation and concerns of academic writing highlight the most significant challenges subjects of many academic papers face. One of the most cited problems is the lack of automation when performing literature reviews. While researchers emphasize the importance of literature review, the fact remains that a researcher has to endure looking at many papers before they find the right one. Fulfilling this requirement delays the other section of research work.

Another problem is the unavailability of personalized features where a user can search and summarize content as necessary. Existing tools are not able to give research-contextual and specific summaries to a particular researcher who has framed questions and seeks information concerning these question.

And, a large number of potential solutions are demanding real-time and multi-scaled performance that most existing approaches cannot offer, thereby failing to deal with a large amount of academic wise data collections or responses that take a long time. The underutilization of Retrieval-Augmented Generation (RAG) in academic research limits the potential to generate real-time, concise summaries from retrieved documents, which this project aims to resolve.

1.3 What is Hybrid RAG?

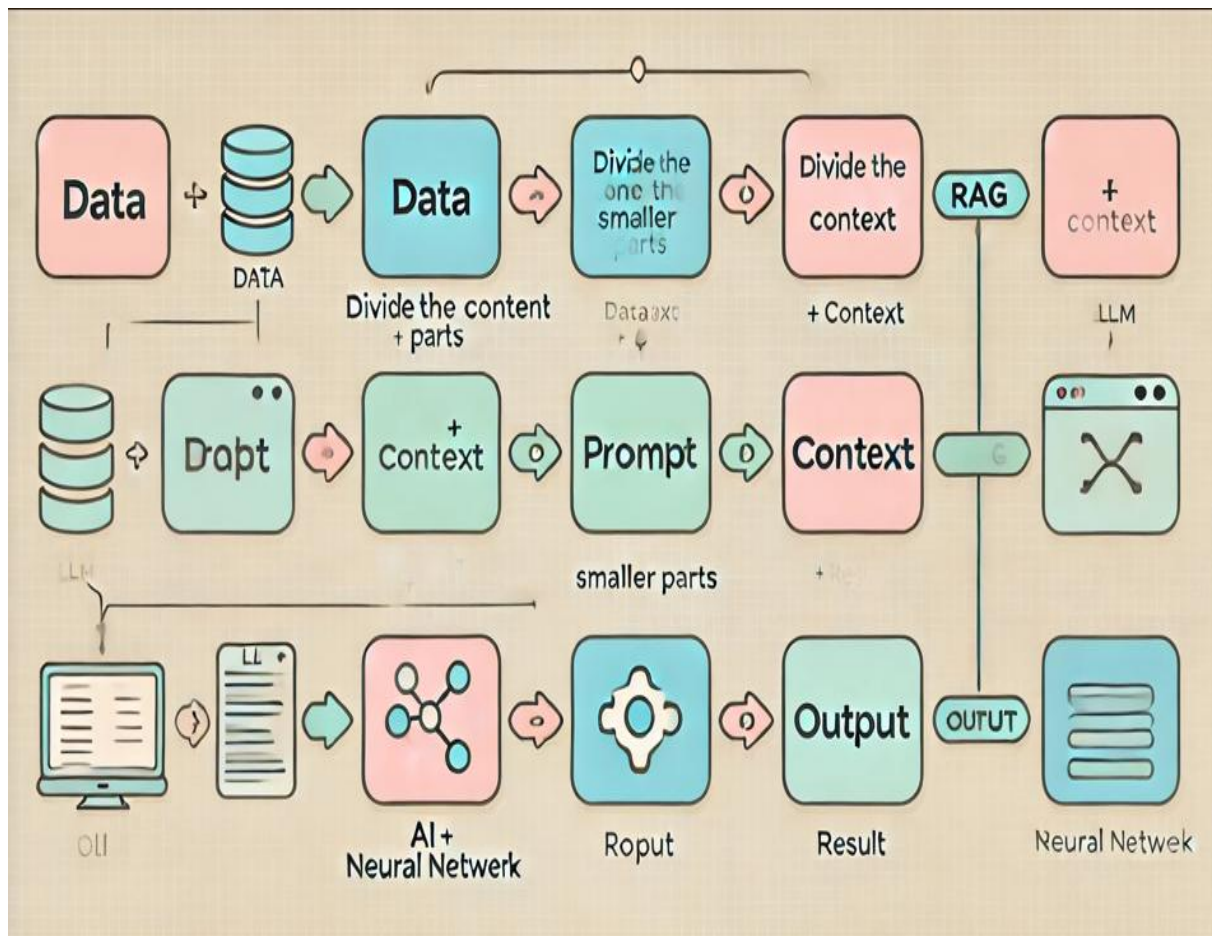


Fig 1.1 Flow Diagram of RAG

Hybrid Retrieval-Augmented Generation (Hybrid RAG) is a sophisticated AI method that improves the retrieval and generation power of Large Language Models (LLMs). Traditional RAG, based on vector-based retrieval, is surpassed by Hybrid RAG, which integrates various retrieval approaches to enhance the accuracy, coverage, and relevance of the information that is recovered. Through the use of both semantic (vector) and symbolic (keyword-based) search, Hybrid RAG makes AI-generated responses more accurate and informative.

Hybrid RAG integrates different retrieval techniques to optimize search results. The two main retrieval methods used are dense retrieval (semantic search) and sparse retrieval (keyword-based search). Dense retrieval uses embeddings to represent text in a high-dimensional space, allowing for similarity searches based on meaning rather

than exact keywords. Models like Sentence Transformers, SBERT, or OpenAI embeddings are commonly used for this approach. Vector databases such as Pinecone, FAISS, Weaviate, and ChromaDB store these embeddings and perform fast similarity searches. Sparse retrieval relies on traditional keyword-matching algorithms like BM25 and TF-IDF. These methods are particularly effective when searching for specific terms in structured text, such as research papers, legal documents, or medical records. Elasticsearch and Whoosh are popular tools for keyword-based search.

One of the key benefits of Hybrid RAG is higher accuracy, as it ensures that the retrieval process captures both semantic meaning and exact term matching. This improves the quality of responses, especially in specialized fields like scientific research, healthcare, and finance. Hybrid RAG also provides broader coverage, as it combines structured and unstructured data sources. This makes it useful for applications that require retrieving text, tables, images, or graphs. Additionally, Hybrid RAG is faster and more efficient, as it balances the strengths of vector search (contextual understanding) with keyword search (precision and recall). This hybrid approach reduces the chances of missing important information while ensuring relevant and context-rich retrieval.

1.4 How Hybrid RAG Works

Hybrid Retrieval-Augmented Generation (Hybrid RAG) is a sophisticated approach that enhances AI-powered information retrieval by combining multiple search techniques. The process involves several key steps, ensuring that the retrieved knowledge is comprehensive, relevant, and contextually appropriate. Below is a detailed breakdown of how Hybrid RAG functions from query input to response generation.

- User Query Processing

The process begins when a user submits a query. This query can be a simple keyword search or a more complex, natural language question. The system preprocesses the query by breaking it down into meaningful components through tokenization. It then applies various natural language processing (NLP) techniques, such as stemming, lemmatization, and stop-word removal, to

enhance the query's effectiveness. Additionally, the system converts the query into embeddings using a neural network model to facilitate vector-based retrieval. By transforming the query into a structured format, the system ensures compatibility across different retrieval mechanisms, optimizing both accuracy and efficiency.

- **Dual Retrieval Mechanism**

Hybrid RAG leverages two primary retrieval techniques to ensure a robust search process:

- **Vector-Based Retrieval (Dense Retrieval):** The system converts the user query into an embedding using a neural network model like BERT, SBERT, or OpenAI embeddings. These embeddings represent the semantic meaning of the query and are then compared with document embeddings stored in a vector database (e.g., Pinecone, FAISS, Weaviate). This allows the system to retrieve documents that are semantically similar to the query, even if they do not share exact keywords.
- **Keyword-Based Retrieval (Sparse Retrieval):** Simultaneously, the query is also processed using traditional keyword-matching techniques such as BM25 or TF-IDF. This method is particularly useful for retrieving documents that contain exact terms matching the query, ensuring precision in cases where specific terminology is critical (e.g., scientific literature, legal documents).

- **Merging and Ranking Results**

Once both retrieval methods have fetched relevant documents, the system merges the results. A ranking mechanism is applied to prioritize documents based on:

- **Relevance Score:** Each document receives a score based on how well it matches the query in both semantic and keyword-based searches.
- **Hybrid Weighting:** A weighted approach combines scores from vector-based and keyword-based retrieval to balance contextual relevance with keyword accuracy.

- **Knowledge Graph Integration (Optional):** If a knowledge graph is available, it helps refine the retrieval by considering relationships between concepts. This improves the contextual understanding and provides additional supporting information.

- **Augmenting the LLM with Retrieved Data**

After ranking, the top-ranked documents are selected and passed as context to a Large Language Model (LLM) such as OpenAI's GPT, Gemini AI, or LLaMA. The LLM processes these documents and generates a response that is factually accurate and contextually relevant.

- **Generating the Final Response**

The LLM synthesizes the retrieved knowledge and constructs a well-formed, informative response. The response may include:

- Direct answers to the user's query.
- Summarized insights from multiple sources.
- References to the original retrieved documents for transparency.

Hybrid Search

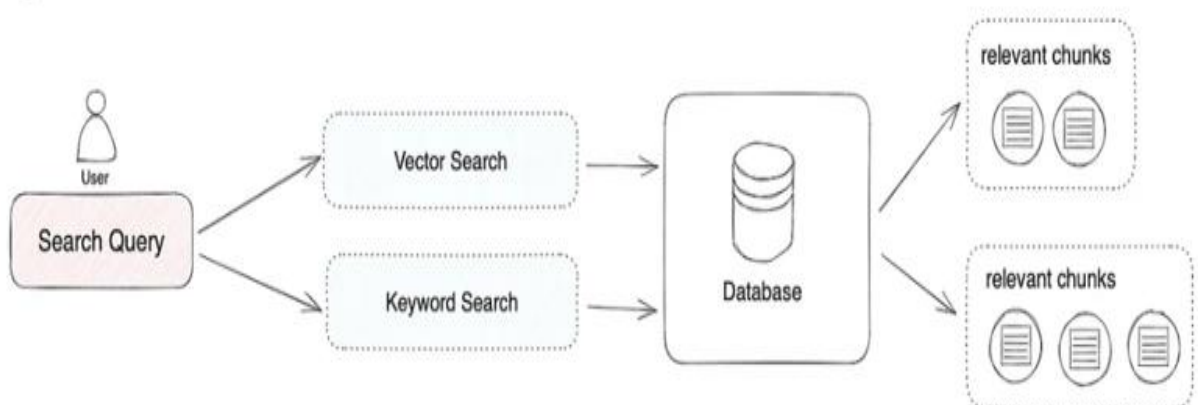


Fig 1.2 Hybrid Search Flow

1.5 Why Hybrid RAG?

Hybrid Retrieval-Augmented Generation (Hybrid RAG) is particularly well-suited for this project because it addresses the limitations of traditional retrieval methods by combining the advantages of both sparse (keyword-based) and dense (semantic-based) retrieval techniques. Traditional keyword-based search methods, such as BM25, rely on exact word matches, which means they can retrieve papers that contain specific keywords but may struggle with synonyms, paraphrased concepts, and contextual variations. This can lead to missing out on relevant research that uses different terminology to describe the same ideas. On the other hand, dense retrieval methods, such as those powered by neural embeddings (e.g., Sentence-BERT or OpenAI embeddings), capture the deeper semantic meaning of text, allowing for more nuanced and conceptually similar matches. However, dense retrieval alone may sometimes fail to retrieve highly relevant documents when the query contains uncommon technical terms or specific jargon that the model has not been trained on extensively. By combining these two approaches, Hybrid RAG ensures a robust and comprehensive retrieval system that effectively balances exact keyword matching with semantic understanding, significantly improving the relevance and accuracy of search results.

In the context of this project, where the goal is to automate the extraction, embedding, and retrieval of research papers, Hybrid RAG plays a crucial role in refining the literature review process. Research papers often contain complex, domain-specific language, and the ability to retrieve documents based on both precise terminology and broader conceptual similarities ensures that users can find the most relevant papers even if different phrasing is used. Additionally, Hybrid RAG enhances the search experience by reducing noise—irrelevant papers that contain the right keywords but lack actual relevance to the user’s intent. This is particularly useful for researchers who need to quickly identify key papers in their field without sifting through numerous irrelevant results.

Furthermore, Hybrid RAG's ability to work alongside a graph database makes it even more powerful in this project. Since research papers have interconnected relationships—such as citation networks, co-authorship patterns, and thematic similarities—storing embeddings and metadata in a graph database allows the system to retrieve and rank papers not just based on textual similarity but also on structural relationships within the research landscape. For instance, if a user searches for a specific

topic, the system can not only retrieve the most semantically relevant papers but also recommend related works that cite or are cited by those papers, enabling a more holistic literature review. This is especially useful for interdisciplinary research, where understanding the connections between different fields is crucial.

Another major advantage of Hybrid RAG in this project is its ability to generate meaningful summaries and insights from retrieved papers. Unlike traditional search engines that merely return a ranked list of documents, Hybrid RAG-powered systems can synthesize information from multiple sources to provide coherent summaries, highlight key findings, and even suggest research gaps. This capability significantly enhances research efficiency by allowing users to grasp the essence of multiple papers without reading each one in full. Additionally, since Hybrid RAG integrates large language models (LLMs) for answer generation, it can refine search queries dynamically, improving iterative literature exploration.

By leveraging Hybrid RAG, this project ensures that researchers have a powerful, intelligent retrieval system that goes beyond conventional keyword-based searches, enabling more precise, efficient, and insightful literature reviews. The combination of keyword and semantic search, graph-based contextual retrieval, and AI-powered summarization makes Hybrid RAG the ideal approach for automating the extraction and analysis of academic research papers. This ultimately enhances research productivity, reduces information overload, and ensures that researchers can access the most relevant studies quickly and effectively.

1.6 Objectives

The objective of this project is to create an automated system that facilitates the process of conducting literature reviews by extracting, converting, and organizing academic research papers from multiple sources. The system will use advanced Retrieval Augmented Generation(RAG) techniques and graph databases to perform semantic searches based on user-defined keywords, retrieve relevant papers, and display them in an organized and easily interpretable format. This project aims to significantly reduce the time and effort required for researchers to manually search, read, and synthesize research papers, while ensuring that the most relevant and useful papers are identified quickly and accurately.

The system will automate the extraction of research papers from various academic databases, such as Google Scholar and Scopus, converting them into machine-readable text. This eliminates the need for researchers to manually download and read each paper, which is often time-consuming. The extracted data will include the full text of the papers along with important metadata such as author names, publication year, and the journal in which the paper was published. This metadata will facilitate efficient organization and retrieval of research papers.

To enhance searchability, each research paper will be transformed into an embedding—a vector representation that captures the semantic meaning of the document. This process will be achieved using advanced NLP techniques such as Sentence-BERT or similar models designed for document embeddings. By converting papers into embeddings, the system will capture the underlying themes and topics, making it possible to perform more accurate searches based on content rather than just keywords.

The embeddings, along with the associated metadata, will be stored in a graph database. This choice of database enables seamless organization and retrieval of research papers based on relationships between them. Graph databases allow the system to model and visualize connections between different papers, such as those authored by the same researchers or those sharing similar topics, methodologies, or findings. This structure improves the discovery of relevant research and provides deeper insights into the relationships between various academic works.

To ensure efficient research, the system will allow users to input specific keywords or phrases related to their research interests. Based on these inputs, a semantic similarity search will be performed across the stored embeddings. Unlike traditional keyword-based search methods, which may return irrelevant results, this system will rank papers based on their semantic similarity to the user's query. This ensures that the retrieved papers are more relevant and aligned with the researcher's focus, minimizing time spent on reviewing unrelated studies.

The retrieved research papers will be displayed in a structured format for easy interpretation. The output will be presented in a table format, including key details such as author names, year of publication, a brief summary of the research findings, and the

objective of the paper. This structured presentation will enable researchers to quickly assess the relevance of each paper without having to read through entire articles, significantly improving the efficiency of the literature review process.

By leveraging advanced NLP techniques and graph databases, the system will enhance the accuracy of paper retrieval, ensuring that researchers can find relevant studies even if different terminology or phrasing is used. The automated process will reduce human error in summarizing and organizing research papers, leading to more consistent and comprehensive literature reviews. This innovation will ultimately improve research efficiency and streamline the discovery of relevant academic work.

1.7 Gap Identification

This system fills a formation gap in the typical literature review process tools as they tend to be little automated. Currently, existing tools of literature review are seen to typically demand a researcher searches for their relevant articles manually. This evidently is not efficient. Therefore, the first gap is the absence of an automated system to scrape research papers from the web and make the process of data collection simpler. This requires user-centric search and summarization. Existing tools hardly provide a personalized summary based on the users' research areas of interest. Thus, the project will provide a system that will fetch relevant papers as well as give tailored summaries to enhance the quality and applicability of the results.

Moreover, embeddings are not used efficiently in the current search tools for academic literature. Even though embeddings are known to enhance the search quality and velocity through context-centric retrieval, the majority of the utilities do not optimize this functionality. This project employs embeddings by means of Sentence-BERT, making it quicker and with better precision in extraction of data. Further on, the issue of scalability is quite prevalent in the existing tools, as a lot of them are not able to process large volumes of data without lags.

This project addresses that disadvantage by employing Pinecone for vector storage that is scalable and real time retrieval that is fast so that huge volumes of data sets can be managed by the system. And last but not the least RAG is the technique that is rarely heard of in the academic domain. This project seeks to fill this void by showing

how RAG can be carried out in the academic space enabling the shift from futuristic, innovative, theoretical AI technologies to their practical usage in research.

1.8 Scope of the Project

The scope of this project centers around the development of an advanced automated system designed to streamline the literature survey process and facilitate efficient research paper retrieval. The system will be capable of extracting research papers from various academic databases, such as Google Scholar, Scopus, and other scholarly repositories, eliminating the need for manual downloads and searches. Once extracted, these research papers will undergo preprocessing, where they will be converted into structured, machine-readable text formats to ensure seamless processing.

A core feature of the system is its ability to transform research papers into vector embeddings using state-of-the-art Natural Language Processing (NLP) models, such as Sentence-BERT or other embedding models specifically trained for academic text. These embeddings serve as high-dimensional numerical representations that capture the semantic meaning of the research content, allowing for more accurate and context-aware retrieval. Unlike traditional keyword-based retrieval systems that rely solely on exact term matches, the use of vector embeddings enables the system to understand and match research papers based on conceptual similarity, even when different terminologies or phrasings are used.

To facilitate efficient storage and retrieval, the generated embeddings, along with important metadata such as title, authors, publication year, and journal name, will be stored in a graph database. This choice of database architecture is particularly advantageous for research paper management, as it allows the system to model and analyze complex relationships between papers. For instance, the graph structure enables linking papers based on shared authorship, citations, similar topics, or overlapping research methodologies. By leveraging the power of graph databases, the system can provide users with not just direct search results but also intelligently recommended papers that are contextually related to their query, enhancing the depth and breadth of the literature review.

One of the key functionalities of the system is the implementation of similarity search techniques that allow users to retrieve research papers based on user-provided keywords or queries. The similarity search mechanism will compare the input query with the stored embeddings, ensuring that retrieved results are not just keyword matches but are ranked based on semantic relevance. This will significantly improve the accuracy of search results, ensuring that researchers can quickly access the most pertinent academic papers relevant to their field of study.

The retrieved papers will be presented in a structured table format, allowing for easy interpretation and comparison. This table will display essential details, including the title of the paper, the names of the authors, the year of publication, a concise summary of the research findings, and the main objective of the paper. This structured presentation will enable researchers to quickly assess the relevance of each paper without having to read through the entire document, thereby saving time and improving efficiency.

By automating the literature survey process, this system aims to transform how researchers and academics discover, organize, and review scholarly publications. The integration of cutting-edge NLP techniques, graph databases, and similarity search algorithms ensures that users can efficiently navigate vast collections of research papers, ultimately enhancing productivity and enabling more informed decision-making in academic and scientific research. The project, therefore, not only streamlines the process of conducting literature surveys but also fosters a more intelligent, data-driven approach to academic research discovery.

CHAPTER 2

LITERATURE SURVEY

2.1 Literature Survey

[1]. This paper provides a comprehensive overview of GraphRAG (Graph-based Retrieval-Augmented Generation) methods, systematically categorizing its workflow into three key stages: Graph-Based Indexing, Graph-Guided Retrieval, and Graph-Enhanced Generation. Graph-Based Indexing structures knowledge into a graph format, organizing entities and their relationships to facilitate efficient retrieval. Graph-Guided Retrieval leverages these structured connections to extract highly relevant and contextually meaningful information, surpassing traditional retrieval methods by incorporating semantic relationships. Graph-Enhanced Generation integrates graph-structured data into Large Language Models (LLMs) to produce more coherent, factually grounded, and explainable responses while mitigating hallucinations. The paper further explores the major technologies underlying GraphRAG, including graph neural networks (GNNs), knowledge graph embeddings, and hybrid retrieval models, along with various training approaches such as supervised fine-tuning, reinforcement learning, and contrastive learning. Additionally, it highlights diverse application areas, including automated literature reviews, question answering, biomedical research, legal analysis, and personalized recommendations. Evaluation methodologies, including benchmarks, metrics, and real-world performance assessments, are also discussed to gauge GraphRAG's effectiveness. Industrial applications showcase its role in enhancing knowledge discovery, decision-making, and AI-driven automation. The paper concludes by identifying future research directions, emphasizing scalability improvements, real-time adaptability, multimodal integration, and reducing computational overhead, ultimately aiming to advance innovation and adoption of GraphRAG in both research and industry.

[2]. This paper introduces a survey on data-augmented LLMs, suggesting a RAG task classification strategy that groups user queries into four levels according to the kind of external data needed. It establishes these query levels, lists pertinent datasets, and explains important challenges and useful techniques. The paper also investigates three key methods of bringing external data into LLMs—context, small model, and

fine-tuning—pointing out their advantages and disadvantages. The aim is to enable readers to comprehend data needs, determine the most important bottlenecks, and create systematic solutions to designing successful LLM applications.

[3] It is very challenging to evaluate retrieval-augmented generation (RAG) systems, and especially their retrieval models. The conventional end-to-end evaluation processes are heavy on resources, and retrieval performance evaluation based on query-document relevance labels has poor correlation with RAG system overall effectiveness. For this purpose, we propose a new evaluation mechanism called eRAG in which every retrieved document is processed individually by the large language model inside the RAG system. The output generated is then evaluated against ground truth labels for the downstream task, making the document's downstream performance its relevance label. With a variety of task-specific metrics, we obtain document-level annotations and combine them with set-based or ranking metrics. Large-scale experiments on multiple datasets demonstrate that eRAG captures a higher correlation with downstream RAG performance than baseline approaches, boosting Kendall's τ correlation by 0.168 to 0.494. In addition, eRAG greatly improves computational efficiency, minimizing runtime and needing as much as 50 times less GPU memory compared to common end-to-end evaluation techniques.

[4] In this paper the author explores the performance of Retrieval-Augmented Generation (RAG) and fine-tuning (FT) in enhancing language models (LMs) for dealing with low-frequency entities in question-answering tasks in this paper. Through large-scale experiments on twelve LMs with various sizes and structures, we compare different fine-tuning, data augmentation, and retrieval methods. Our findings indicate that although FT improves performance for entities of varying levels of popularity, RAG largely surpasses FT, especially for the least popular factual knowledge. Additionally, both methods are helped by improvements in retrieval and data augmentation techniques. While fine-tuning is beneficial for smaller models, it is computationally intensive. To alleviate this, we propose Stimulus RAG, a new strategy that outperforms fine-tuning-based solutions, avoiding expensive data augmentation and fine-tuning while successfully enhancing LMs with less frequent factual knowledge.

[5] Retrieval-Augmented Generation (RAG) is an innovative AI method that supercharges tasks by delivering accurate and current external knowledge. In the age of AI-Generated Content (AIGC), the capacity of RAG to retrieve purposeful information greatly enhances the output quality of generative AI. Although Large Language Models (LLMs) have shown phenomenal breakthroughs in language comprehension and generation, they remain plagued by issues like hallucinations and stale internal knowledge. To meet these challenges, Retrieval-Augmented Large Language Models (RA-LLMs) have been introduced, which incorporate external authoritative sources of knowledge to support generation quality over and above the model's internal abilities. In this survey, we give a systematic overview of RA-LLMs from three major technical aspects: architectures, training methods, and applications. We start with a brief overview of the basic ideas and recent developments in LLMs. We then discuss mainstream research in RA-LLMs systematically, comparing their architectures, training methods, and real-world applications and elaborating on the problems they solve. We conclude by discussing current limitations and identifying promising research directions to further develop the field.

[6]The author, offers an overview of the inclusion of Retrieval-Augmented Generation (RAG) in Artificial Intelligence Generated Content (AIGC). Advances in model algorithmics, foundational models, and high-quality corpora have boosted the development of AIGC, but its improvement is plagued with issues like updates in knowledge, management of long-tail data, stopping data leakage, and high computation expenses. These concerns are answered by RAG with the use of an information retrieval step to enhance content production through applicable extraneous data that serves to reinforce accuracy and solidity. They classify RAG methods according to the manner in which retrieval mechanisms complement the generation process, formulating a cohesive outlook that emphasizes important developments and essential technologies. We further synthesize multiple improvements that refine RAG deployment, enhancing its effectiveness in real-world usage. On a different plane, we discuss applications of RAG across multiple modalities and tasks and provide useful recommendations for researchers and practitioners. In addition, we also address benchmarking frameworks,

discuss the shortcomings of existing RAG systems, and suggest fruitful areas for future work.

[7] Information retrieval (Ir) systems like search engines have become for each individual the ideal means of retrieving information and play an indispensable role in dialogue systems, question-answering, and recommender platforms. Over the years, it has transformed from traditional term-based methods to very advanced neural models that tend to capture more complex contextual-semantic relationships. But, these neural models still suffer from data sparsity, interpretability limitations, generation of plausible-sounding but incorrect answers, etc. It, therefore, requires adopting a hybrid combination of traditional sparse retrieval for efficiency along with more modern neural architectures, making possible a deeper linguistic understanding. It is the advent of Llm's like Chatgpt and GPT-4 that invaded natural language processing with their distinctive abilities to read, write and reason text, hence generated greater and greater interest in adapting LLMs with IR. Given the speed at which research is developing in this area, it has become almost urgent to consolidate all germane methodologies and provide systematic analysis. This survey, thus, surveys what one can say as at the intersection of LLMs and IR, focusing on certain principal components such as query rewriting, retrieval models, reranking techniques, and reading mechanisms. It will also show emerging research directions such as building smart search agents for future opportunities in this evolving area.

[8] This research examines the ability of Retrieval-Augmented Generation (RAG) in large language models (LLMs) to automate systematic literature reviews (SLRs). RAG-infused LLMs have the potential to automate data extraction, summary generation, and trend identification. Though they possess high-end language generation and understanding capabilities, LLMs are dependent on pre-trained, static knowledge, which can cause errors and hallucinations. RAG overcomes these shortcomings by integrating the generative power of LLMs with real-time information retrieval for enhanced accuracy and reliability. We give a detailed description of the three main components of the RAG framework—retrieval, augmentation, and generation—after which we present its applications to automating SLRs. Finally, we sketch some of the most important future research directions, such as incorporating domain-specific

LLMs, multimodal data processing, and using multiple retrieval sources. We introduce a RAG-based LLM architecture for SLR automation, involving four important phases: literature search, literature screening, data extraction, and information synthesis. Future work will be directed toward maximizing interactions between LLM choice, training methodologies, RAG methods, and prompt engineering to maximize retrieval from single scientific documents. The objective is to produce detailed outputs covering areas like up-to-date research status, available knowledge gaps, and future trends.

[9] Large language models (LLMs) with external data have been found to perform outstandingly on real-world tasks. Methods such as Retrieval-Augmented Generation (RAG) and fine-tuning are being researched more and more for incorporating external data into LLMs. Using these data-augmented models effectively across domain-specific areas, though, is a challenging problem. These problems vary from fetching the right information and accurately interpreting user intent to leveraging LLMs' reasoning abilities for complex tasks. We contend that there is no single way to use data-augmented LLMs because problems frequently arise from a failure to correctly identify the essential focus of the task or in needing to isolate several interdependent capabilities to better solve problems. In this survey, we present a categorization scheme for RAG tasks, placing user queries into four broad levels according to the type of external data needed and the main goal of the task: explicit fact queries, implicit fact queries, interpretable rationale queries, and hidden rationale queries. We formalize these query levels, present respective datasets, and summarize the key challenges together with the best techniques to overcome them. We also consider three main ways of incorporating external data into LLMs—context-based augmentation, small model integration, and fine-tuning—and compare their respective strengths, weaknesses, and best applications. This research intends to enable readers to comprehend the data requirements and most important issues in constructing LLM applications in a structured way, providing guidance and tactics for maximizing their deployment.

[10] This work introduces AutoSurvey, a speedy and formal way of automating the construction of inclusive literature surveys, especially for quickly evolving domains such as artificial intelligence. Conventional survey composition is undermined by overwhelming amounts and sophistication of information, requiring more effective methods. Although large language models (LLMs) have solutions to offer, they are limited by issues such as narrow context windows, limitations in parametric insight, and a lack of standardized assessment benchmarks. AutoSurvey addresses these problems using a formalized workflow involving four stages: retrieval and outline creation, drafting subsections using expert LLMs, merging and

polishing, and strict evaluation with iterative enhancement. The major contributions of our work are a full solution to automated literature surveys, an effective evaluation protocol, and experiment results demonstrating AutoSurvey's superiority.

[11] In the Industry 4.0 age, artificial intelligence (AI) is capable of augmenting decision-making and problem-solving in engineering and technical endeavors, but its application in Industrial design—where one aims to maximize product functionality, value, and form—is still untapped. With this gap in mind, in this study, an unsupervised learning-driven review approach is suggested, employing k-means clustering to determine prominent journals and articles, and subsequent co-citation, bibliographic coupling, and co-occurrence analysis to inspect the relevant literature. There are six broad clusters of research listed, such as supply chain lenses on product innovation, manufacturability in product development, intelligent design and engineering tools, service innovation using applied AI, Industry 4.0 technologies in manufacturing, and blockchain-based AI applications. Document summarization and assessment help assign names to these clusters, with future research pathways focusing on sustainable design, trusting AI, and integrating emerging technology to develop the next generation of AI-driven industrial design.

[12] This paper introduces a machine learning-based system designed to semi-automate the literature review process. By leveraging machine learning algorithms, the system evaluates whether scientific papers are relevant to a given research topic. The user interface facilitates the literature review through a structured workflow, including data source selection, query formulation, topic searches, paper retrieval, selection of relevant studies, machine learning-based classification, and final paper display and export. A comparative analysis between manual and automated reviews assesses differences in efficiency, accuracy, and usability. Findings suggest that the highest sensitivity and accuracy are achieved through a hybrid machine learning model that integrates multiple unweighted algorithms. Cross-validation across two datasets revealed consistency in hyperparameters, demonstrating the system's potential for broader application in automated literature reviews.

[13] Artificial intelligence (AI) has become highly significant in modern computing, excelling in complex tasks traditionally performed by humans. It enables knowledge representation, inference, text processing, and data-driven learning, making it particularly useful for automating labor-intensive tasks such as scientific literature analysis. Conducting a systematic literature review (SLR) manually is time-consuming and requires careful planning, literature search, analysis, and result reporting.

Depending on the research field, the sheer volume of retrieved papers—often in the hundreds or thousands—makes filtering relevant studies and extracting key insights a challenging and error-prone process. However, many of these tasks are repetitive and well-suited for AI-driven automation. This paper presents a survey of AI techniques developed over the past 15 years to assist researchers in conducting systematic literature analyses. It examines supported tasks, applied algorithms, and available tools from 34 primary studies while offering a historical perspective on the field's evolution and the role of human intervention in an increasingly automated SLR process.

[14] Recommender systems play a crucial role in managing information overload across various online services. With the rise of Large Language Models (LLMs), significant progress has been made toward developing next-generation recommender systems. However, LLM-based recommenders face challenges due to hallucinations, outdated knowledge, and limited domain-specific understanding, which stem from their underlying LLM architectures. Retrieval-Augmented Generation (RAG) has gained attention as a solution by integrating external knowledge sources to enhance LLM performance. Despite its promise, standard RAG methods often introduce noise and fail to capture structural relationships within knowledge, reducing their effectiveness in recommendation tasks. To overcome these challenges, we propose a knowledge graph (KG)-enhanced retrieval framework, K-RagRec, which retrieves high-quality, up-to-date structural information from KGs to improve recommendation generation. Through extensive experiments, we demonstrate the effectiveness of our approach in enhancing LLM-based recommender systems.

[15] Large Language Models (LLMs) excel in reasoning tasks but face challenges such as hallucinations and outdated knowledge. To mitigate these issues, Knowledge Graph (KG)-based Retrieval-Augmented Generation (RAG) incorporates structured external knowledge to enhance LLM responses. However, existing KG-based RAG methods often struggle to balance retrieval effectiveness and efficiency, making it difficult to extract an optimal amount of relevant information for LLM processing. In this study, we introduce SubgraphRAG, an enhanced KG-based RAG framework that retrieves subgraphs and utilizes LLMs for reasoning and answer generation. Our approach incorporates a lightweight multilayer perceptron combined with a parallel triple-scoring

mechanism to enable efficient and adaptable subgraph retrieval. Additionally, we integrate directional structural distances to improve retrieval effectiveness. The framework allows for dynamic adjustment of subgraph sizes based on query requirements and LLM capabilities, maintaining a balance between model complexity and reasoning ability. This design ensures a scalable and generalizable retrieval process. Notably, smaller LLMs such as Llama3.1-8B-Instruct achieve competitive, explainable reasoning, while larger models like GPT-4o deliver state-of-the-art accuracy—all without requiring fine-tuning. Comprehensive evaluations on the WebQSP and CWQ benchmarks confirm that SubgraphRAG significantly improves efficiency, accuracy, and reliability by minimizing hallucinations and enhancing response grounding.

2.2 Findings of the literature survey

Traditional methods of conducting literature reviews have evolved significantly with the introduction of AI and RAG techniques. These techniques leverage external knowledge to enhance the relevance, accuracy, and contextual understanding of results. A significant focus of the research is on using LLMs for data retrieval and integration with structured knowledge such as knowledge graphs. Multiple papers review how RAG integrates with LLMs to provide improved results in terms of information retrieval, recommendations, and systematic literature reviews. Methodologies such as graph-based retrieval, machine learning, and knowledge graph augmentation are highlighted as ways to address challenges like noise in retrieval, system complexity, and limited multi-modal capabilities.

This survey emphasize the categorization of user queries, advanced retrieval algorithms, and integration with minimal training adjustments. Additionally, the survey highlights practical applications of these techniques, from semi-automated literature surveys to AI-generated surveys and industrial design reviews. Limitations such as noise in retrieval results, increased system complexity, and the gap between retrievers and generators remain areas for further research. Future directions include improving multilingual capabilities, trustworthy retrieval, and more efficient handling of external knowledge sources. Some of the important findings from the literature surveys are:

Graph-Based Retrieval in RAG:

- Papers such as Graph Retrieval-Augmented Generation and Knowledge Graph Retrieval-Augmented Generation for LLM-based Recommendation emphasize the use of knowledge graphs to enhance retrieval processes.
- Graph-based systems address noise, improve accuracy, and ensure better contextual relevance in results.

Query Categorization Frameworks:

- Research papers classify user queries into distinct categories (e.g., explicit fact queries, rationale queries) to develop customized retrieval and generation solutions.
- This categorization framework simplifies the integration of external data and improves task-specific retrieval.

Challenges and Limitations:

- Common challenges include noise in retrieval results, the gap between retrievers and generators, and increased system complexity.
- Papers such as Retrieval Augmented Generation for AI-Generated Content outline these issues and propose solutions such as better augmentation methodologies.

Future Scope:

- Researchers identify areas such as multi-lingual capabilities, trustworthy RA-LLMs, and multi-modal RAG systems as potential future advancements.
- There is also a need for more detailed studies on integrating RAG with LLMs for systematic literature reviews and industrial applications.

Applications of RAG:

- Applications include automated literature surveys, AI-generated systematic reviews, and industrial design research.
- Tools like AutoSurvey demonstrate how LLMs can generate survey outlines, drafts, and refined outputs automatically.

Comparative Studies:

- Studies such as Fine Tuning vs. Retrieval Augmented Generation for Less Popular Knowledge analyze the effectiveness of RAG techniques versus traditional fine-tuning methods.

Enhancements in Systematic Literature Reviews:

- Papers like Automating Systematic Literature Reviews with Retrieval-Augmented Generation highlight how RAG-based systems improve literature reviews by integrating real-time retrieval with generative models.

Benchmarking and Validation:

- Comprehensive evaluations on benchmarks like WebQSP and CWQ validate the effectiveness of frameworks like SubgraphRAG in improving efficiency, accuracy, and response grounding.

CHAPTER 3

DISSERTATION DISCRIPTION AND GOALS

3.1 Existing System

Existing automated literature survey tools, such as Google Scholar, PubMed, and specialized platforms like Mendeley and EndNote, are widely used for searching, organizing, and managing academic papers. These systems provide users with access to vast databases of scholarly articles, enabling efficient citation tracking, reference management, and the use of basic search filters to refine queries. Some platforms even offer features like article recommendations based on user interests or trending research in particular fields, helping users stay updated on the latest developments. Despite these advantages, most of these tools rely heavily on keyword-based search methods, which can sometimes result in broad or irrelevant results if the chosen keywords are not precise enough or fail to capture the full context of the research query. This can lead to the retrieval of unrelated papers that do not meet the user's needs. Additionally, some platforms incorporate machine learning algorithms to recommend relevant papers, but these algorithms often lack the precision needed to provide highly accurate or topic-specific suggestions, leading to a less effective literature review process. Furthermore, the issue of "hallucination" of irrelevant results can still arise, where the system might generate suggestions that appear plausible but are disconnected from the user's specific research interests or query, diminishing the reliability of these tools.

3.2 Disadvantages of Existing System

- **Keyword-Based Search Limitations:**Traditional search engines and academic databases primarily rely on keyword-based search techniques. While keywords can provide quick access to related documents, they have several limitations when it comes to handling complex queries. Keywords often fail to capture the nuance and intent behind a user's question, leading to results that may be loosely related or even irrelevant. For instance, if a user searches for "machine learning in healthcare," the system might return papers containing the keywords "machine" and "healthcare," but they might not necessarily address the user's

specific interest in the application of machine learning techniques within the healthcare domain. Additionally, keyword searches cannot handle synonyms, paraphrases, or contextual variations, making them less effective for retrieving papers that might use different terminology or phrasing to describe similar concepts. Consequently, this leads to information overload and missed relevant results.

- **Lack of Semantic Understanding:**Existing search systems largely focus on keyword matching without understanding the meaning behind the words. This means they do not perform semantic searches, which would involve interpreting the context, intent, and relationships between terms in a query. For example, a user searching for "impact of AI on healthcare outcomes" may not receive relevant results if the system only looks for papers with the exact phrase "AI on healthcare outcomes" but misses related terms like "artificial intelligence," "healthcare improvements," or "patient outcomes." Without semantic understanding, the system struggles to capture the broader or deeper meanings behind the search, and this limitation prevents it from retrieving papers that might be closely related but don't exactly match the user's query. The lack of semantic capabilities also means that the system cannot learn from or adapt to the context of the user's interests, further reducing the relevance of search results.
- **No Built-in Summarization:**Traditional literature search platforms typically do not offer built-in summarization features, forcing users to read through entire research papers to extract key insights. Given the vast number of research papers published every day, this can be an overwhelming and time-consuming task for researchers. Especially when dealing with lengthy and highly technical papers, summarizing key points manually can take up significant amounts of time, leaving little space for actual analysis or application of the knowledge. Summarization tools powered by AI or NLP techniques can significantly improve the efficiency of the literature review process by providing users with concise summaries of papers, highlighting the key findings, methodologies, and conclusions. This could save researchers valuable time, allowing them to

quickly assess whether a paper is worth reading in full or if it can be excluded from further consideration.

- **Limited Customization and Filtering:** While many academic search engines and tools provide basic filtering options such as date range, document type, or author, these filters often lack flexibility and depth. Researchers may need to perform more nuanced filtering based on a variety of factors, such as specific research methods, statistical techniques, or even paper quality, but existing systems typically do not offer the customization necessary for this. For instance, a researcher might want to narrow down their search to only peer-reviewed articles using machine learning techniques in a particular area of medicine, but current filtering systems often do not allow for such specific criteria. The inability to tailor search results to these highly specific needs leads to a less efficient review process and may leave users with irrelevant or incomplete results. A more customizable filtering system would help users refine their search results more effectively and obtain exactly the type of information they require.
- **Scalability Issues with Large Datasets:** As academic databases grow exponentially with the increasing volume of research publications, traditional search systems can struggle to scale effectively. Many platforms experience performance degradation when handling large datasets or when regular updates are made to include new papers. This can result in slower retrieval speeds, with systems taking longer to process user queries and deliver relevant results. In some cases, the growing size of databases can lead to less accurate results, as the system might not be able to efficiently retrieve the most relevant papers from a larger pool of documents. Scalability issues also become problematic when there are constant updates to the dataset, such as new publications being added daily. A system that is unable to handle the dynamic nature of the research environment may produce outdated or incomplete results, frustrating researchers who need access to the latest research in their field.

- **Minimal Personalization:** Most current academic search platforms do not adapt to the unique preferences or research needs of individual users. These systems typically provide a one-size-fits-all approach, where the search results are not customized to reflect the user's specific research interests, previous searches, or reading habits. As a result, users may have to sift through irrelevant or low-priority papers in order to find those that are most pertinent to their research question. A more personalized search experience could save researchers time by learning from their past behavior, preferences, and search patterns, and offering recommendations based on those insights. For example, a researcher focused on medical AI applications might benefit from a system that prioritizes papers in their field, highlights recent advancements, or provides suggestions based on related topics. Without personalization, users are left to manually sort through results, which can detract from the efficiency and value of the search process.

3.3 Proposed System

To address the limitations of manual literature reviews, this project suggests an automatic research paper retrieval system that combines Neo4j (graph database) and Weaviate (vector database) through a Hybrid Retrieval-Augmented Generation (RAG) process. The system would improve research paper retrieval through the integration of structured knowledge graphs with semantic search capabilities in order to offer correct, context-aware, and complete results to researchers. By automating the data storage, retrieval, and collection processes, this system saves considerable time and effort in carrying out literature reviews without missing any relevant papers.

1. Data Collection and Preprocessing

The system collects research papers from various sources such as Scopus, IEEE Xplore, arXiv, and other academic databases. Information is retrieved with the help of APIs and web scraping methods, obtaining metadata (title, authors, year, citations, keywords) and full-text content whenever it is available. The extracted information is further preprocessed with the help of Natural Language Processing (NLP) methods like tokenization, stopword removal, stemming, and text normalization in order to have

clean and organized input for embedding generation and construction of knowledge graphs.

2. Graph-Based Knowledge Organization (Neo4j)

Neo4j is the graph database, where papers are nodes, and their relations—citations, co-authorship, keyword similarities, and topic categories—are edges. The graph-based architecture allows researchers to conduct complex queries, such as finding highly cited papers, emerging research topics, important authors in a field, and paper clusters based on shared citations. Through the use of Neo4j's graph traversal algorithms, the system returns contextually specific papers through direct and indirect connections, presenting a networked view of research trends.

3. Semantic Search Using Weaviate

Aside from structured retrieval, Weaviate is employed as a vector database to hold text embeddings of research papers. Using state-of-the-art NLP models like Sentence-BERT or OpenAI's embedding models, each research paper's title, abstract, and keywords are converted into high-dimensional vector representations. When a user enters a query, the system converts it into an embedding and performs a similarity search in Weaviate, retrieving semantically similar papers—even those that do not contain exact keyword matches. This enables researchers to find papers that are conceptually similar but may have employed alternative terminologies.

4. Hybrid Retrieval-Augmented Generation (RAG) Approach

In order to achieve maximum retrieval precision, this project utilizes a hybrid RAG mechanism that integrates structured search from Neo4j with semantic search from Weaviate. When a user makes a query, the system initially queries Neo4j to retrieve papers based on direct relationships (e.g., author collaborations, citations) and then fine-tunes the results with Weaviate's similarity search.

This two-stage retrieval approach ensures that users receive both highly relevant and contextually meaningful papers. The results retrieved are shown in a tabular structured

format consisting of titles of papers, authors, publication years, abstracts, and research purposes, and their best delivery is a well-organized easy-to-browse literature review experience. By combining graph-based and vector-based retrieval, this system to be proposed improves research efficiency, accuracy, and accessibility, and makes literature reviews faster, more comprehensive, and data-driven. This system greatly benefits researchers, students, and professionals who require rapid access to relevant and high-quality academic papers.

3.4 Advantages of the Proposed System

- Automated and Efficient Literature Retrieval**
The system eliminates the need for manual searching by automatically retrieving research papers from multiple sources, significantly reducing the time and effort required for a literature review. This automation helps researchers focus on analyzing insights rather than spending hours searching for relevant papers.
- Hybrid Search for Improved Accuracy**
By integrating Neo4j (graph-based retrieval) and Weaviate (vector-based retrieval), the system ensures highly accurate and contextually relevant results. This Hybrid Retrieval-Augmented Generation (RAG) approach combines structured citation-based search with semantic similarity search, improving retrieval precision and ensuring no relevant paper is missed.
- Better Relationship Mapping Between Papers**
The graph database (Neo4j) enables structured representation of research papers by mapping relationships between authors, citations, keywords, and topics. This helps researchers identify trends, influential authors, and highly cited papers in their field, allowing for better knowledge discovery and network analysis of research domains.
- Semantic Search for Concept-Based Matching**
Unlike traditional keyword-based searches, Weaviate's vector database uses NLP embeddings to find semantically similar papers, even if they do not contain exact keyword matches. This helps researchers discover relevant papers that might use different terminologies, improving the quality and depth of the literature review.

- **Structured and Organized Output**

The retrieved research papers are displayed in a structured tabular format with key details such as title, author, year, abstract, and research objective. This well-organized presentation makes it easier for researchers to analyze and compare papers efficiently.

- **Scalable and Adaptable System**

The system can be expanded to accommodate additional research databases, enhanced NLP models, and new data sources. This scalability ensures that the system remains relevant and useful as academic literature continues to grow.

- **Saves Time and Reduces Human Effort**

By automating data retrieval, storage, and summarization, this system significantly reduces the workload for researchers, students, and professionals, making the literature review process faster, more systematic, and less error-prone.

Overall, this proposed system enhances research efficiency, ensures comprehensive paper retrieval, and provides a more insightful and data-driven literature review process, making it an invaluable tool for academics, scholars, and professionals.

3.5 Use Case Scenario

A PhD researcher is working on a survey paper about recent advancements in air quality monitoring using AI. Traditionally, they would manually search various databases like Google Scholar, IEEE Xplore, and Scopus, filtering through thousands of papers using different keywords. This process is not only time-consuming but also risks missing semantically relevant papers due to variations in terminology.

Using the Dynamic Research Paper Retrieval System, the researcher simply enters a topic or keywords (e.g., "AI for Air Quality Prediction"). The system then:

- Extracts and organizes relevant research papers from multiple sources.

- Retrieves papers based on structured queries (e.g., finding top-cited papers and author networks).
- Fetches semantically similar papers even if they do not contain exact keywords.
- Displays results in a tabular format with titles, authors, abstracts, and objectives.

This hybrid RAG approach ensures the researcher quickly identifies the most relevant papers, saving weeks of manual effort and improving the quality and depth of their literature review.

CHAPTER 4

DESIGN APPROACH AND DETAILS

4.1 Detail design approach

The Hybrid RAG-Based Automated Literature Review Assistant is designed to streamline and enhance the process of conducting a literature review by integrating structured databases, vector embeddings, and generative AI models. The system aims to provide accurate, contextually relevant, and well-structured literature reviews by leveraging graph-based knowledge retrieval and semantic search using vector embeddings. This project is built around three core functionalities: data collection and storage, hybrid retrieval and ranking, and AI-driven summarization and presentation.

1. Data Collection and Storage

The system will automatically fetch research papers from multiple sources, including Scopus API, ArXiv API, and OpenAlex API, as well as allow users to upload their own PDFs. Metadata such as title, authors, publication year, abstract, keywords, DOI, and citation count will be extracted and stored in a structured database (PostgreSQL or MongoDB). The full text of the research papers will be preprocessed, removing special characters, normalizing text, and segmenting it into meaningful chunks. These chunks will then be converted into embeddings using Sentence-BERT (SBERT) or OpenAI embeddings and stored in a vector database (Pinecone or Weaviate) for efficient semantic search. Additionally, a graph database (Neo4j or ArangoDB) will be used to store relationships between research papers, authors, keywords, and journals, enabling network-based retrieval. The graph database schema will include nodes such as Papers, Authors, Keywords, and Journals, and relationships like AUTHOR_OF, CITES, TAGGED_WITH, and BELONGS_TO, allowing structured traversal of academic literature.

2. Hybrid Retrieval and Ranking

The core retrieval mechanism of the system follows a Hybrid RAG approach, combining graph-based retrieval and vector search to ensure both precision and contextual relevance. When a user enters a keyword-based search query, the system

first queries the graph database to find directly related papers and their citation network. This ensures retrieval of not only the requested papers but also influential papers in the same domain. Simultaneously, the system performs a semantic similarity search in the vector database, retrieving documents that have conceptual relevance to the query, even if they do not contain exact keyword matches. These two retrieval results are then combined using hybrid ranking mechanisms, which may include BM25-based weighting for keyword relevance and cosine similarity scores for embedding-based retrieval. The highest-ranked research papers are then selected for further processing, ensuring that the user gets both directly relevant and semantically related literature.

3. AI-Driven Summarization and Presentation

Once the most relevant papers have been retrieved, the system leverages LLM-based summarization (OpenAI GPT-4, Gemini, or Mistral) to generate concise and structured summaries. The AI models extract key insights such as research objectives, methodologies, findings, and conclusions, ensuring that the user gets a well-structured understanding of the selected papers. The final output is presented in a tabular format, displaying Serial Number, Paper Title, Authors, Year of Publication, Summary, and Research Objective, allowing users to quickly scan and understand the findings. Additionally, a PDF viewer will be integrated into the system, enabling users to read the full text of selected papers without leaving the interface. The results can also be exported in multiple formats, such as CSV, Word, or PDF, allowing seamless integration into research workflows.

4. User Interface and Workflow

The system will be deployed as a Streamlit-based web application with an intuitive and user-friendly interface. The user starts by entering a search query (keywords, research area, or author name), and the system executes the hybrid RAG retrieval process in the backend. The results are displayed in a structured table, along with graphs and visualizations that highlight citation networks, keyword trends, and paper clusters. The user can refine their search, filter results by publication year, author, or journal, and download summaries for reference. The backend is powered by FastAPI, ensuring fast response times and easy API integration. Additionally, users can interact with an AI

chatbot to ask for deeper explanations, trend analysis, or personalized literature recommendations.

5. Future Enhancements

To further improve the system, predictive analysis can be integrated, using time-series forecasting models (such as LSTM or ARIMA) to predict research trends in a particular field. Citation impact analysis can be implemented to rank papers based on their influence in the research community. Finally, mobile and voice-based interfaces can be added for more accessible literature searches, allowing researchers to query the system using natural language.

4.2 System Architecture

System Flow:

- Data Collection: Extract research papers from external sources.
- Preprocessing and Embedding: Clean and vectorize the paper data.
- Vector Storage: Store embeddings in a graph database for similarity-based retrieval.
- User Query: Accept user queries and perform similarity search.
- Result Retrieval and Display: Return relevant papers in a structured format.

This system will streamline the process of conducting literature surveys by automating the retrieval of pertinent research papers and presenting them in an easily accessible format. It will save researchers significant time in identifying relevant literature based on specific research interests.

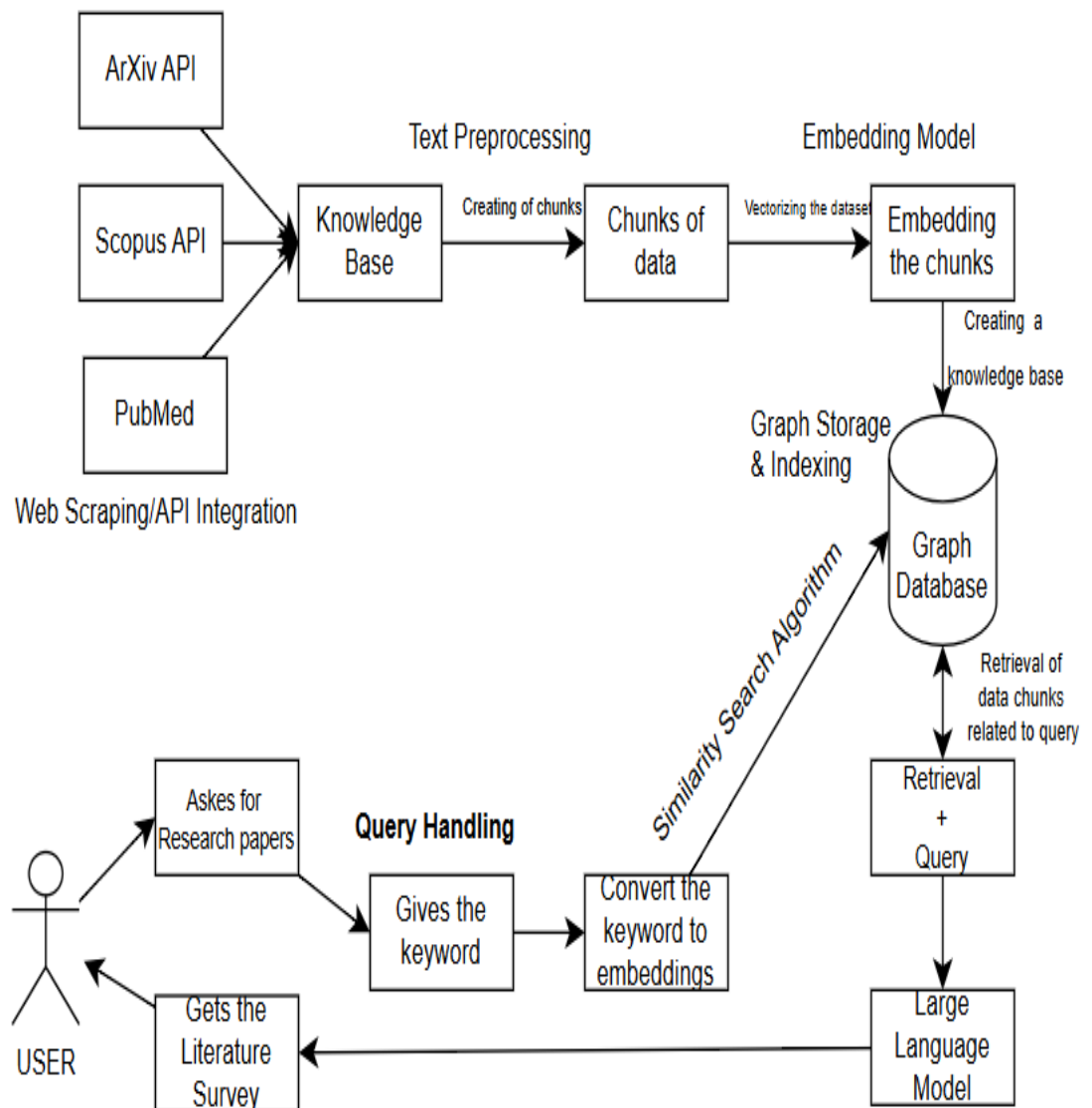


Fig 4.1 System Architecture

4.3 UML DIAGRAMS:

Class Diagram

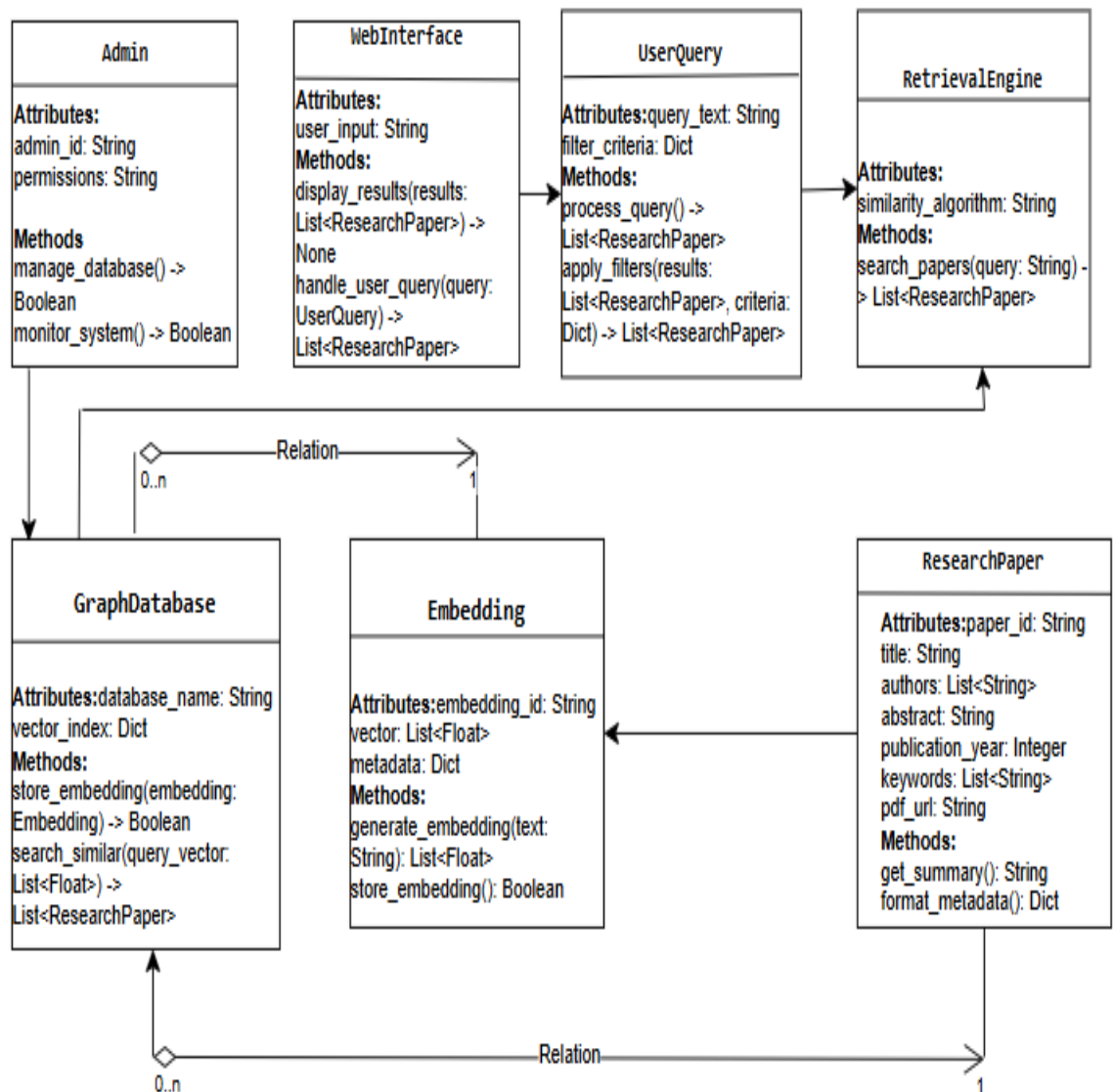


Fig 4.2 Class Diagram

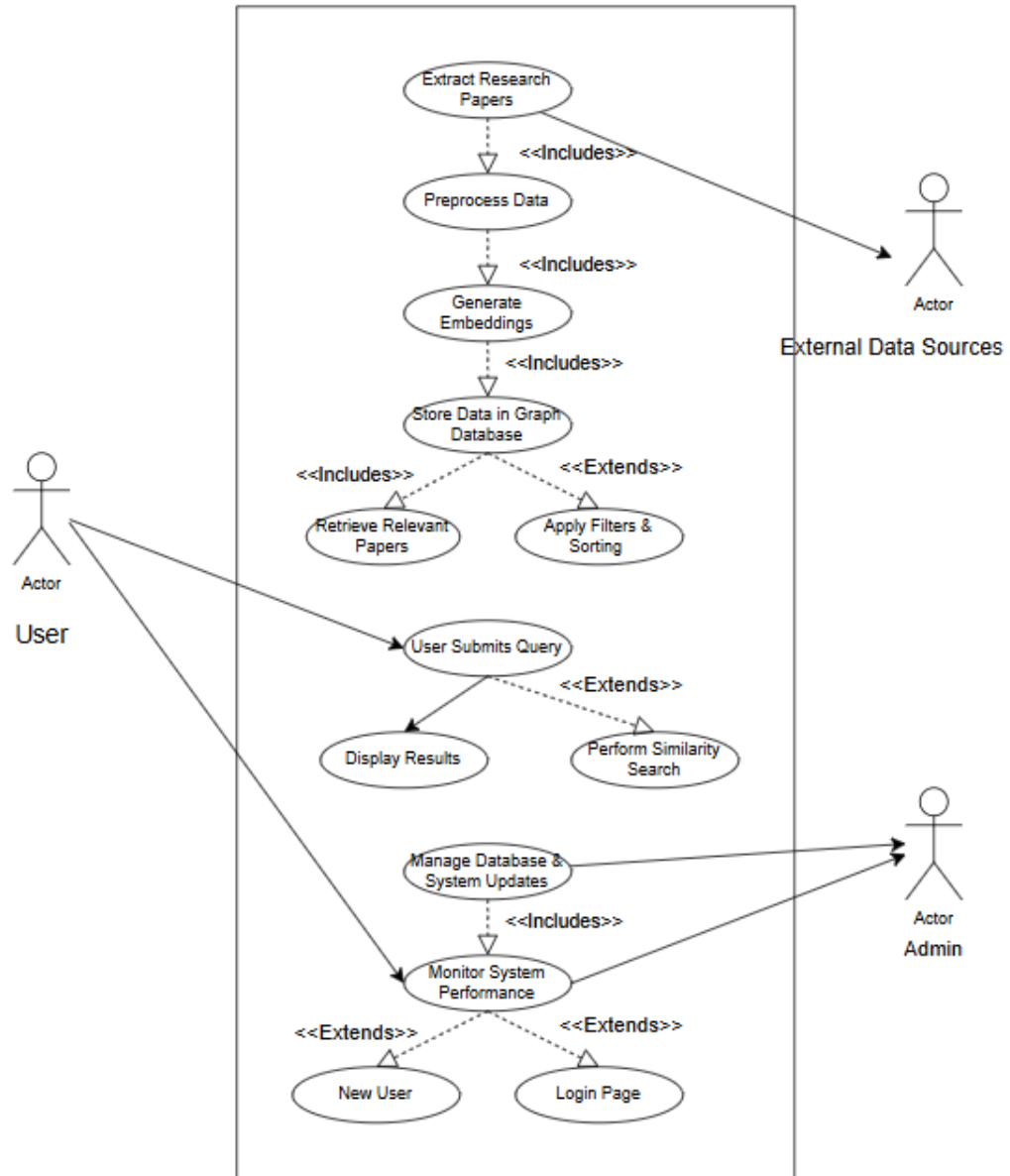


Fig 4.3 Use Case Diagram

4.4 Constraint of the dissertation

1. Data Accessibility and Licensing Restrictions

One of the primary challenges in implementing a Hybrid RAG-based literature review system is the accessibility of research papers. Many high-quality research databases, such as Scopus, IEEE Xplore, and Elsevier, have restricted access policies that require expensive subscriptions or impose strict rate limits on API usage. This limitation affects the system's ability to retrieve comprehensive and up-to-date research papers, leading to gaps in the literature review. While open-access repositories like ArXiv and

OpenAlex provide free access, they do not cover all fields equally, which may result in incomplete or biased retrieval.

2. Computational Costs and Scalability Issues

Running a hybrid retrieval system that integrates vector search (semantic similarity) and graph-based search (citation networks) demands high computational resources. Vector databases like Pinecone and Weaviate require GPU acceleration for efficient search operations, and large-scale graph traversal in databases like Neo4j can be computationally expensive as the dataset grows. Additionally, LLM-based summarization of long research papers significantly increases API costs, making it challenging to scale the system cost-effectively. Without optimization, real-time retrieval and summarization could introduce significant delays, reducing the system's usability.

3. Accuracy and Reliability of Summarization

A major drawback of using Large Language Models (LLMs) for summarization is their tendency to hallucinate information, meaning they may generate misleading or incorrect summaries. Since LLMs rely on pre-trained knowledge and the retrieved document's content, any misinterpretation of scientific language or missing contextual information can lead to inaccurate conclusions. Moreover, summarizing complex research papers into concise yet informative summaries is difficult, as important details may be lost in compression. This limitation affects the credibility of the automated literature review system and necessitates human verification.

4. User Query Formulation and Retrieval Relevance

The effectiveness of the literature review system depends heavily on how users formulate their search queries. Users may struggle to find the right combination of keywords and context, leading to irrelevant or overly broad search results. Furthermore, hybrid retrieval systems weigh semantic similarity and citation-based relationships differently, which may result in suboptimal ranking of retrieved papers. If the system fails to correctly balance keyword matching, vector similarity, and citation relationships, it may return results that are not the most relevant, reducing its efficiency in assisting researchers.

4.5 Methodology

The methodology of this project is designed to systematically extract, process, embed, and retrieve research papers using a hybrid Retrieval-Augmented Generation (RAG) approach. This combines vector-based search, keyword-based search, and graph-based relational retrieval to ensure that the most relevant and contextually rich research papers are presented to the user. The methodology is divided into five key phases: Data Collection and Preprocessing, Embedding Generation and Storage, Query Processing and Similarity Search, Retrieval-Augmented Generation (RAG) for Summarization, and User Interaction and Filtering.

- Data Collection and Preprocessing

The first step in the pipeline involves collecting and preprocessing research papers to create a structured dataset for retrieval. Research papers are sourced from multiple academic repositories such as arXiv, Scopus, IEEE Xplore, Springer, and OpenAlex. Where available, APIs are used to fetch metadata and full-text documents. In cases where APIs are not available, web scraping techniques are implemented to extract relevant data. Once the research papers are collected, preprocessing is performed to prepare them for embedding generation. This involves several steps:

- Text Extraction: Libraries like PyMuPDF or pdfminer are used to extract text from PDF documents. This ensures that the full textual content of the research paper is available for processing.
- Cleaning: Unnecessary elements such as references, tables, and special characters are removed to retain only the useful content. This step is crucial to prevent noise from affecting the quality of embeddings.
- Metadata Processing: Essential attributes such as title, authors, publication year, keywords, abstract, and objectives are extracted. These attributes facilitate structured search and improve retrieval efficiency by allowing additional filters.

By applying these preprocessing techniques, the system ensures that the dataset is well-structured and optimized for retrieval and embedding generation.

- Embedding Generation and Storage

Once the research papers have been preprocessed, the next step is to convert their content into vector representations that enable similarity-based retrieval.

This process involves text chunking, tokenization, embedding generation, and hybrid storage in a graph database.

- **Chunking and Tokenization:** Since large research papers cannot be embedded as a single unit due to token limits in embedding models, they are broken into manageable chunks using frameworks like LangChain. This ensures that even smaller sections of a paper can be retrieved based on semantic similarity.
- **Embedding Model Selection:** The text chunks are then converted into high-dimensional vector embeddings using models such as Sentence-BERT (SBERT) or OpenAI's text embedding models. These embeddings capture the semantic meaning of the text and enable efficient similarity search.
- **Graph-Based Storage:** Unlike traditional vector databases that store embeddings in an isolated manner, this project leverages graph-based storage using Neo4j or Weaviate. These databases not only store the vectors but also maintain relationships between research papers based on citations, co-authorship, and keyword similarities. This structure enhances retrieval by incorporating both semantic and relational context.

This hybrid storage mechanism combines vector search for semantic similarity, keyword-based indexing for exact matching, and graph-based storage for relational retrieval, making the system more accurate and contextually aware.

- **Query Processing and Similarity Search**

Users interact with the system through a user-friendly web interface (built using Streamlit or FastAPI) where they can input keywords, research topics, or specific questions. The system then processes the user query to identify and retrieve the most relevant research papers using a hybrid approach.

- **Vectorization of User Queries:** The input query is converted into an embedding using the same model used for document embeddings. This ensures that the semantic meaning of the query is captured.
- **Hybrid Similarity Search:** The retrieval process combines three approaches:
 - **Vector Search:** Using cosine similarity or k-nearest neighbors (k-NN) to retrieve papers that are semantically similar to the query.

- Keyword-Based Search: Matching user queries with indexed metadata (titles, abstracts, and keywords) to retrieve papers based on exact matches.
- Graph-Guided Retrieval: Enhancing the search using graph relationships, where papers that are cited together or have similar authorship and research topics are prioritized.

By integrating vector search, keyword matching, and graph-based retrieval, the system ensures that users receive the most comprehensive and relevant results based on their queries.

- Retrieval-Augmented Generation (RAG) for Summarization

Once relevant research papers have been retrieved, the system employs Retrieval-Augmented Generation (RAG) to generate concise, query-specific summaries using Large Language Models (LLMs) such as GPT-4 or Gemini. The RAG approach improves the summarization process by leveraging factual data from retrieved documents, reducing the chances of hallucination by LLMs.

- Generating Structured Summaries: The retrieved documents are processed to extract key components such as research objectives, methodologies, findings, and limitations. This ensures that the user receives structured insights without having to read the full paper.
- Query-Specific Summarization: Instead of generating generic summaries, the LLM adapts its output based on the user's specific question or research interest. For example, if the user queries "What are the recent advancements in AI-based medical diagnosis?" the system generates summaries that highlight only the relevant aspects of retrieved papers.

This Hybrid RAG approach ensures that summaries are not only accurate but also contextually relevant, making the literature review process faster and more effective.

- User Interaction and Filtering

The final step involves presenting the retrieved and summarized research papers in a structured format through a web-based interface that enhances usability. Users can interact with the system in multiple ways:

- **Sorting and Filtering:** Users can refine their search results based on publication year, authors, relevance score, and journal/conference type. This helps them narrow down the most relevant papers for their research.
- **Interactive Exploration:** Users can iteratively refine their queries, view related papers based on graph connections, and explore trends in the research domain. The system may also provide recommendations based on past queries and retrieved papers.

By incorporating these user-friendly features, the system ensures that researchers can efficiently discover, explore, and analyze relevant research papers, significantly reducing the time required for literature reviews.

CHAPTER 5

TECHNICAL SPECIFICATIONS

5.1 Dataset Collection

The data collection process for this project involves automated extraction of research papers from multiple online sources using web scraping and APIs. First, relevant academic databases such as Scopus, IEEE Xplore, and arXiv are accessed via their APIs or web scraping techniques to gather metadata, including titles, authors, abstracts, publication years, and citations. The system ensures data accuracy by filtering and removing duplicate entries while also standardizing formats for consistency. Once the metadata is collected, the full-text content of the papers is extracted wherever possible. Next, Natural Language Processing (NLP) techniques are applied to preprocess the text, including tokenization, stopword removal, and text normalization, preparing the data for embedding generation. Each research paper is then converted into a vector representation using a pre-trained embedding model like Sentence-BERT or OpenAI's embedding models. These vectorized embeddings are stored in Weaviate, enabling semantic search capabilities, while structured metadata, including citation networks and author relationships, is stored in a graph database for relational querying. This dual-database approach ensures that users can retrieve papers either through semantic similarity matching or by exploring structured connections, significantly enhancing the efficiency and depth of literature review processes.

5.2 Neo4j Graph Database

In this project, Neo4j is used as the graph database to organize and store structured relationships between research papers, authors, keywords, and citations, enabling advanced query-based retrieval. When research papers are collected from sources like Scopus, IEEE Xplore, and arXiv, their metadata—including title, authors, publication year, citations, and keywords—is stored in Neo4j as nodes and edges. For instance, each paper is represented as a node, connected to author nodes, keyword nodes, and citation nodes, forming an interconnected knowledge graph. This structured representation allows users to perform complex queries, such as finding the most highly

cited papers on a topic, identifying key researchers in a field, or discovering papers connected through common citations. By integrating Neo4j with Weaviate, the system benefits from both structured relationship-based retrieval and semantic search. When a user searches for a research topic, the hybrid system first queries Neo4j to identify relevant papers based on author collaborations, co-citations, and keyword relations, then refines the results using Weaviate's vector-based semantic matching. This hybrid RAG approach enhances literature retrieval by combining structured academic knowledge with deep semantic understanding, offering researchers a comprehensive and context-aware literature review experience.

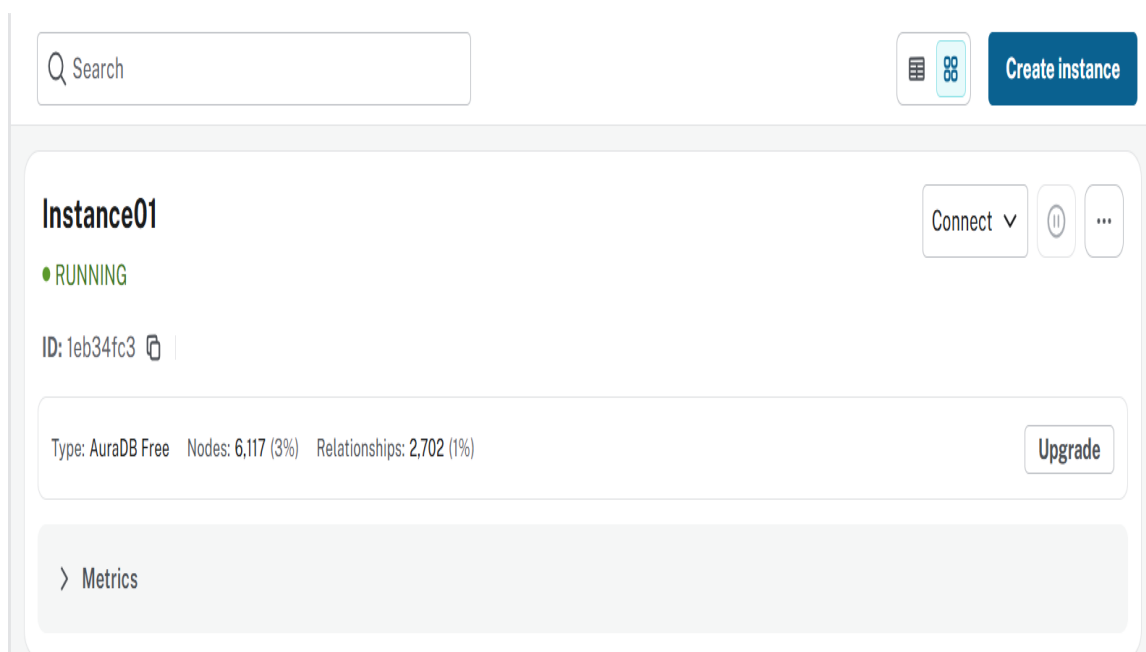


Fig 5.1 Neo4j Instance

5.3 Vector Embeddings

The extracted data is embedded using Sentence-BERT, to facilitate efficient retrieval. A model designed to generate high-quality sentence embeddings. After preprocessing, which includes text cleaning and chunking of abstracts or summaries, each chunk is passed through Sentence-BERT. This model converts the text into dense vector representations that capture the semantic meaning of each passage. Unlike traditional word embeddings, Sentence-BERT fine-tunes the embeddings specifically for sentence-level similarity, making it ideal for tasks involving textual similarity and

retrieval. These embeddings, now in vector form, are stored in a vector database like Pinecone, where they serve as searchable entries.

5.4 Weaviate Database

In this project, Weaviate is used as the vector database to store embeddings of research papers, enabling efficient semantic search and retrieval. Research papers are first processed using state-of-the-art NLP models like Sentence-BERT or OpenAI's text embeddings, which convert the text—such as the title, abstract, and keywords—into high-dimensional vector representations. These embeddings, along with metadata such as the paper's title, authors, publication year, and keywords, are stored in Weaviate using a predefined schema. When a user queries the system with a research topic or a specific question, Weaviate converts the query into an embedding and performs a similarity search, retrieving the most relevant papers even if they do not contain exact keyword matches. This ensures that users get semantically related research articles that might otherwise be overlooked in traditional keyword-based searches. To further enhance retrieval accuracy, the system integrates Weaviate with a graph database that captures structured relationships between research papers, citations, and authors. This hybrid approach combines structured knowledge with deep semantic understanding, allowing users to perform more sophisticated and context-aware literature searches. With Weaviate's efficient indexing and fast querying capabilities, this system significantly reduces the time and effort required for literature reviews, helping researchers, students, and professionals find high-quality academic papers with greater ease and precision.

5.5 Summary And Objective Generation

After retrieving relevant research papers based on their user query, the next step is to generate summaries and objective using the Gemini AI model. The papers contain abstract, author's name and year of publication. This input is then sent to Gemini AI through an API request, where the model processes the text to create a clear, concise summary that captures the main points. Gemini AI's advanced text generation capabilities enable it to identify and producing summaries that are informative. This automated summarization helps users quickly assess the relevance of each paper, streamlining the literature review process.

5.6 Output Of The System

The output of this system is displayed in a Streamlit app, including basic HTML components for a neat and interactive user experience. Streamlit allows for straightforward integration of HTML to format text and create interactive elements that enhance the presentation of each paper. For each retrieved document, the app displays the title, authors, publication date, a concise summary and objective of the paper generated by Gemini AI. Using HTML components, sections are formatted with headings and paragraphs. This approach makes the literature review tool both visually appealing and user-friendly, helping users easily get the papers.

CHAPTER 6

HARDWARE AND SOFTWARE REQUIREMENTS

For the successful implementation of this project, a combination of hardware and software resources is required to support data collection, storage, processing, and retrieval. Below are the detailed requirements:

6.1 Hardware Requirements

A. Computing Power

- Processor: Intel Core i7 (10th Gen or higher) / AMD Ryzen 7 or above
 - Required for efficient data processing, NLP model execution, and query handling.
- RAM: Minimum 16GB (Recommended: 32GB for large-scale datasets)
 - Needed to handle multiple research paper embeddings and graph database queries.
- Storage: Minimum 512GB SSD (Recommended: 1TB SSD + 2TB HDD)
 - SSD for fast indexing and quick retrieval of embeddings and graph relations.
 - HDD for long-term storage of extracted papers and metadata.
- GPU (Optional but Recommended): NVIDIA RTX 3060 (or higher)
 - Required for deep learning models if advanced NLP techniques are used for embeddings.
 - Helps in faster processing of text embeddings with transformers.

B. Network and Server Requirements

- Stable Internet Connection (Minimum 50 Mbps, Recommended: 100+ Mbps)
 - Required for retrieving research papers from online databases.

6.2 Software Requirements

A. Operating System

- Windows 10/11, macOS, or Linux (Ubuntu 20.04 or above)

- Linux is preferred for better performance and server deployment.

B. Programming Language

- Python 3.9 or higher
 - Used for data extraction, processing, and NLP-based embeddings.
 - Libraries: pandas, numpy, scikit-learn, langchain, transformers.

C. Databases

- Neo4j (Graph Database)
 - Stores relationships between papers, authors, keywords, and citations.
 - Enables complex query-based retrieval.
- Weaviate (Vector Database)
 - Stores embeddings of research papers for semantic similarity search.
 - Enables efficient retrieval based on contextual meaning.
- Web Scraping and API Integration
 - BeautifulSoup, Scrapy, or Selenium
Purpose: Used for scraping research papers from sources like arXiv, IEEE Xplore, and Scopus.
- Natural Language Processing (NLP) Frameworks
 - Gemini AI API (Optional for LLM-based summarization)
Purpose: Used for automatic summarization and abstract generation.
- Development Frameworks
 - Streamlit Frontend Dashboard
Purpose: Used for building an interactive web interface for literature review.

CHAPTER 7

DISSERTATION DEMONSTRATION

7.1 Code to Extract The Data And Save In The Pdf

```
import re
# Split papers based on numbering (assuming format is consistent)
papers = re.split(r"\d+\.\s", text)[1:] # Skipping the first split
as it will be empty

# Define regex patterns
title_pattern = r"^(.*?)\n"
authors_pattern = r"\*\*Authors:\*\*\s*(.*?)\n"
year_pattern = r"\*\*Year:\*\*\s*(\d{4})"
summary_pattern = r"\*\*Summary:\*\*\s*(.*)"

# Extract structured data
paper_list = []

for paper in papers:
    title_match = re.search(title_pattern, paper)
    authors_match = re.search(authors_pattern, paper, re.DOTALL)
    year_match = re.search(year_pattern, paper)
    summary_match = re.search(summary_pattern, paper, re.DOTALL)

    title = title_match.group(1).strip() if title_match else None
    authors = authors_match.group(1).split(", ") if authors_match
    else []
    year = int(year_match.group(1)) if year_match else None
    summary = summary_match.group(1).strip() if summary_match else
    None

    if title and year and authors and summary:
        paper_list.append({"title": title, "year": year, "authors":
        authors, "summary": summary})

# Print extracted data
for p in paper_list:
    print(f"Title: {p['title']}, Year: {p['year']}, Authors: {'',
    '.join(p['authors'])}")
```

7.2 Code To Save The Data In Graph Database

```
from neo4j import GraphDatabase

# Update these values from your Neo4j Aura credentials
NEO4J_URI = "neo4j+s://1eb34fc3.databases.neo4j.io" # Get this from
Neo4j Aura
NEO4J_USER = "neo4j" # Default: neo4j
NEO4J_PASSWORD = "xQymX5LbmH91F8HCCLssebH_55NAjVUNz3sbYgBV0cA" #
Your database password
```

```

# Connect to Neo4j Aura
driver = GraphDatabase.driver(NEO4J_URI, auth=(NEO4J_USER,
NEO4J_PASSWORD))

# Function to create nodes and relationships
def add_paper(tx, title, year, summary, authors):
    query = """
    MERGE (p:Paper {title: $title, year: $year, summary: $summary})
    """
    tx.run(query, title=title, year=year, summary=summary)

    for author in authors:
        query = """
        MERGE (a:Author {name: $author})
        MERGE (p)-[:AUTHORED_BY]->(a)
        """
        tx.run(query, title=title, author=author)

# Load extracted papers (from previous extraction step)
def store_papers_in_neo4j(papers):
    with driver.session() as session:
        for paper in papers:
            session.write_transaction(add_paper, paper["title"],
paper["year"], paper["summary"], paper["authors"])

# Close connection
def close():
    driver.close()

# ✓ Example: Store Papers (Use paper_list from the extracted data)
store_papers_in_neo4j(paper_list)

print("Data successfully stored in Neo4j Aura!")

```

7.3 Code To Save The Data In Vector Database

```

import re
from PyPDF2 import PdfReader
import json

# Load the PDF file
pdf_path = "Research_Papers.pdf"
reader = PdfReader(pdf_path)

# Extract text from all pages
extracted_text = "\n".join([page.extract_text() for page in
reader.pages if page.extract_text()])

# Debug: Check extracted text
if not extracted_text.strip():
    print("Error: No text extracted from PDF. Check the file
format.")
    exit()

```

```

# Split based on numbering (e.g., "1.", "2.")
papers_raw = re.split(r'\n\d+\.\s', extracted_text)

# List to store extracted papers
papers = []

for paper_text in papers_raw[1:]: # Ignore first split (before
first paper)
    lines = paper_text.split("\n")

    # Extract title (first line)
    title = lines[0].strip()

    # Extract authors as a string
    authors_match = re.search(r"\s*\s*Authors:\s*(.*?)\s*\s*Year:\s*\s*",
paper_text, re.DOTALL)
    authors = authors_match.group(1).strip() if authors_match else
"Unknown"

    # Extract year
    year_match = re.search(r"\s*\s*Year:\s*(\s*\s*\s*(\d{4}))", paper_text)
    year = int(year_match.group(1)) if year_match else None

    # Extract summary
    summary_match = re.search(r"\s*\s*Summary:\s*(.*?)", paper_text,
re.DOTALL)
    summary = summary_match.group(1).strip() if summary_match else
"No summary found."

    # Store data in dictionary
    paper_dict = {
        "title": title,
        "authors": authors, # Keeping as a string
        "year": year,
        "summary": summary
    }
    papers.append(paper_dict)

# Print formatted JSON output
print(json.dumps(papers, indent=4))

```

7.4 Code To Retrieve The Data Using Hybrid Rag

```

import streamlit as st
import pandas as pd
import weaviate
from neo4j import GraphDatabase
from weaviate.classes.init import Auth

# ----- CONFIGURATIONS -----

```

```

WEAVIATE_URL = "https://f483mnmbrqieikhjrftw.c0.us-east1.gcp.weaviate.cloud"
WEAVIATE_API_KEY = "YDgDDZoaI9GAued6xLMiAWDFro89ZI7v7XJ6"

NEO4J_URI = "neo4j+s://1eb34fc3.databases.neo4j.io"
NEO4J_USER = "neo4j"
NEO4J_PASSWORD = "xQymX5LbmH91F8HCCLssebH_55NAjVUNz3sbYgBV0cA"

# ----- CONNECT TO WEAVIATE -----
client = weaviate.connect_to_weaviate_cloud(
    cluster_url=WEAVIATE_URL,
    auth_credentials=Auth.api_key(WEAVIATE_API_KEY),
)

# ----- CONNECT TO NEO4J -----
neo4j_driver = GraphDatabase.driver(NEO4J_URI, auth=(NEO4J_USER,
NEO4J_PASSWORD))

# ----- FUNCTION TO SEARCH WEAVIATE -----
def search_weaviate(user_query, top_k=5):
    collection = client.collections.get("DemoCollection")

    response = collection.query.near_text(
        query=user_query,
        limit=top_k
    )

    papers = []
    if response.objects:
        for result in response.objects:
            papers.append({
                "title": result.properties.get("title", "N/A"),
                "authors": result.properties.get("authors", "N/A"),
                "year": result.properties.get("year", "N/A"),
                "summary": result.properties.get("summary", "N/A")
            })

    return papers

# ----- FUNCTION TO SEARCH NEO4J -----
def search_neo4j(title):
    query = """MATCH (p:Paper)
        WHERE toLower(p.title) CONTAINS toLower($title)
        RETURN p.title AS cited_title, p.year AS cited_year
        ORDER BY p.year DESC
        LIMIT 5;"""

    with neo4j_driver.session() as session:
        result = session.run(query, title=title)
        papers = [{"cited_title": record["cited_title"],
"cited_year": record["cited_year"]} for record in result]

    return papers

```

```

# ----- HYBRID RAG FUNCTION -----
def hybrid_rag_search(user_query):
    weaviate_results = search_weaviate(user_query)

    for paper in weaviate_results:
        paper["citations"] = search_neo4j(paper["title"])
        if not isinstance(paper["citations"], list):
            paper["citations"] = []

    data = []
    for idx, paper in enumerate(weaviate_results, 1):
        citations_text = ", ".join(
            [f"{c['cited_title']} ({c['cited_year']})" for c in
            paper.get("citations", [])]
        ) if paper.get("citations") else "No citations"

        data.append([
            idx,
            paper.get("title", "Unknown"),
            paper.get("authors", "Unknown"),
            paper.get("year", "Unknown"),
            paper.get("summary", "No summary available"),
            citations_text
        ])

    df = pd.DataFrame(data, columns=["S.No", "Title", "Authors",
    "Year", "Summary", "Citations"])
    return df

# ----- STREAMLIT APP -----
st.set_page_config(page_title="Hybrid RAG Research Assistant",
    layout="wide")

st.markdown("<h1 style='text-align: center; color: #4A90E2; font-size: 40px;'>📖 Hybrid RAG Research Assistant</h1>",
    unsafe_allow_html=True)
st.markdown("<h3 style='text-align: center;'>Enter a research query to find relevant papers from Weaviate & Neo4j</h3>",
    unsafe_allow_html=True)

# User Input
query = st.text_input("🔍 Enter your research query:",
    value="Artificial Intelligence", help="Enter a topic or keyword related to research papers")

if st.button("Search 🔍"):
    with st.spinner("Fetching research papers... 🔄"):
        results_df = hybrid_rag_search(query)

    if results_df.empty:
        st.warning("⚠️ No papers found. Try a different query.")
    else:
        # Enlarged Table Styling

```

```

        st.markdown("""
<style>
.styled-table {
    width: 100%;
    border-collapse: collapse;
    font-size: 18px;
    text-align: left;
}

.styled-table th, .styled-table td {
    border: 1px solid #ddd;
    padding: 15px;
    text-align: left;
    white-space: normal;
}

.styled-table th {
    background-color: #3A3A3A;
    color: white;
    font-size: 20px;
}

.styled-table tr:nth-child(even) {
    background-color: #292929;
}

.styled-table tr:hover {
    background-color: #3C3C3C;
}

/* Fixed Column Widths */
.styled-table td:nth-child(1) { width: 20%; } /* Title */
.styled-table td:nth-child(2) { width: 20%; } /* Authors */
.styled-table td:nth-child(3) { width: 10%; } /* Year */
.styled-table td:nth-child(4) { width: 40%; } /* Summary */
.styled-table td:nth-child(5) { width: 10%; } /* Citations */

/* Fixed Row Height */
.styled-table td {
    height: 100px; /* Set a fixed row height */
    overflow: hidden;
}

/* Scrollable Table */
.table-container {
    overflow-x: auto;
}
</style>
""", unsafe_allow_html=True)

# Generate rows for each paper
# Loop through DataFrame to create rows
for _, row in results_df.iterrows():
    st.markdown(

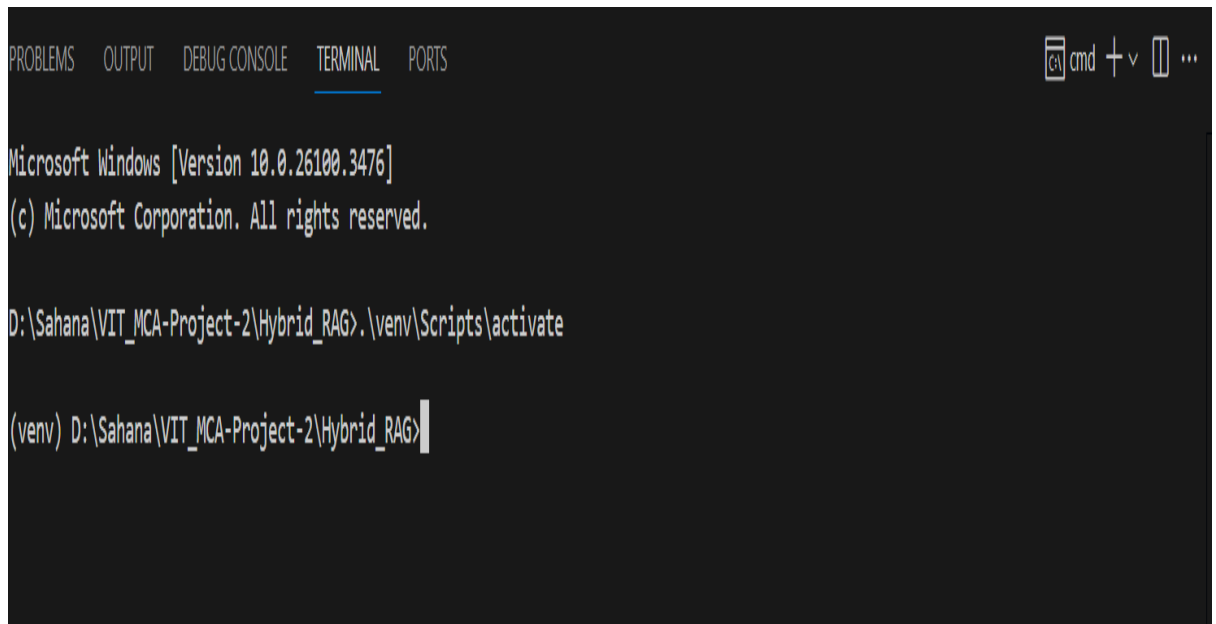
```

```

f"""
<tr>
    <td>{row['Title']}

```

7.5 Output of the system



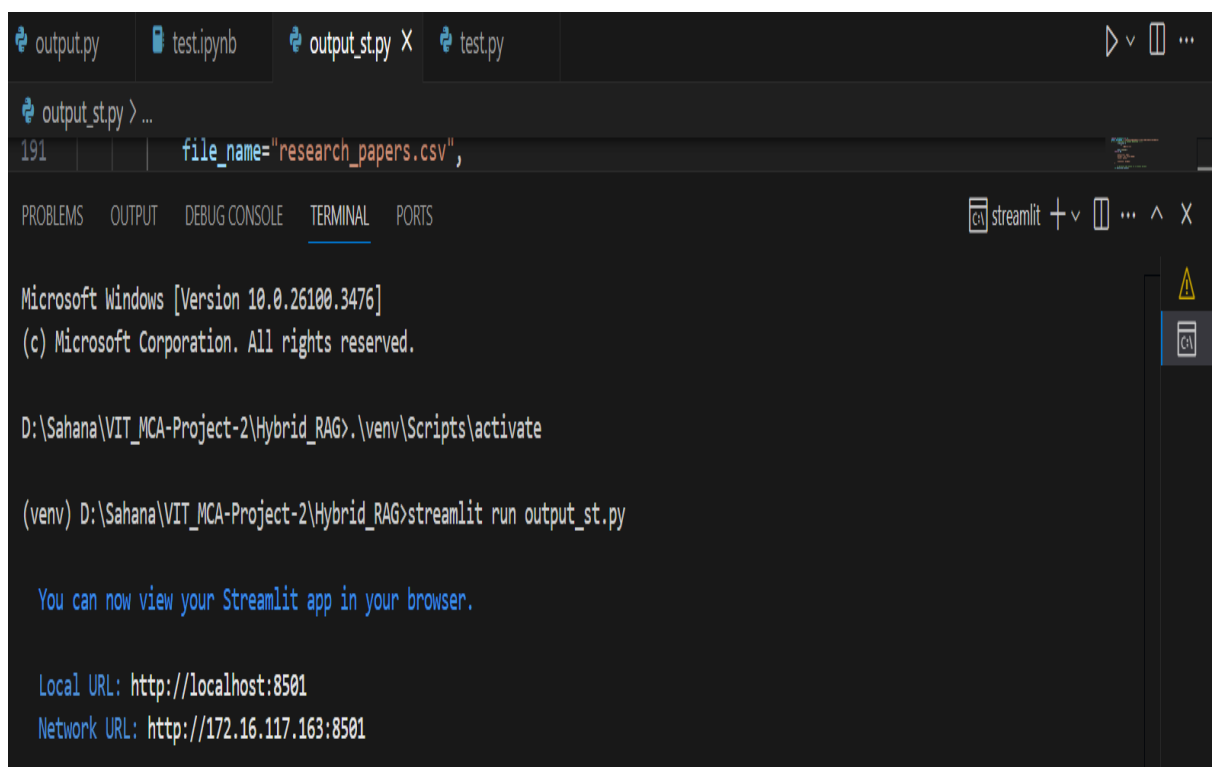
A screenshot of a Windows terminal window. The title bar shows tabs for 'PROBLEMS', 'OUTPUT', 'DEBUG CONSOLE', 'TERMINAL' (which is active), and 'PORTS'. In the top right corner, there are icons for a command prompt, a plus sign, a minus sign, and a three-dot menu. The terminal text shows the Windows version (10.0.26100.3476), copyright information for Microsoft Corporation, the current directory path 'D:\Sahana\VIT_MCA-Project-2\Hybrid_RAG', the command to activate the virtual environment '.\venv\Scripts\activate', and the resulting prompt '(venv) D:\Sahana\VIT_MCA-Project-2\Hybrid_RAG>'.

```
Microsoft Windows [Version 10.0.26100.3476]
(c) Microsoft Corporation. All rights reserved.

D:\Sahana\VIT_MCA-Project-2\Hybrid_RAG>.\venv\Scripts\activate

(venv) D:\Sahana\VIT_MCA-Project-2\Hybrid_RAG>
```

Fig.7.1 Virtual Environment Creation



A screenshot of a Windows terminal window with a code editor open above it. The code editor shows a Python file 'output_st.py' with a line of code: 'file_name="research_papers.csv",' at line 191. The terminal window has the same title bar as Fig. 7.1. The terminal text shows the activation of the virtual environment, followed by the command 'streamlit run output_st.py'. It then displays the message 'You can now view your Streamlit app in your browser.' and the local and network URLs: 'http://localhost:8501' and 'http://172.16.117.163:8501'.

```
Microsoft Windows [Version 10.0.26100.3476]
(c) Microsoft Corporation. All rights reserved.

D:\Sahana\VIT_MCA-Project-2\Hybrid_RAG>.\venv\Scripts\activate

(venv) D:\Sahana\VIT_MCA-Project-2\Hybrid_RAG>streamlit run output_st.py

You can now view your Streamlit app in your browser.

Local URL: http://localhost:8501
Network URL: http://172.16.117.163:8501
```

Fig.7.2 Running Streamlit app

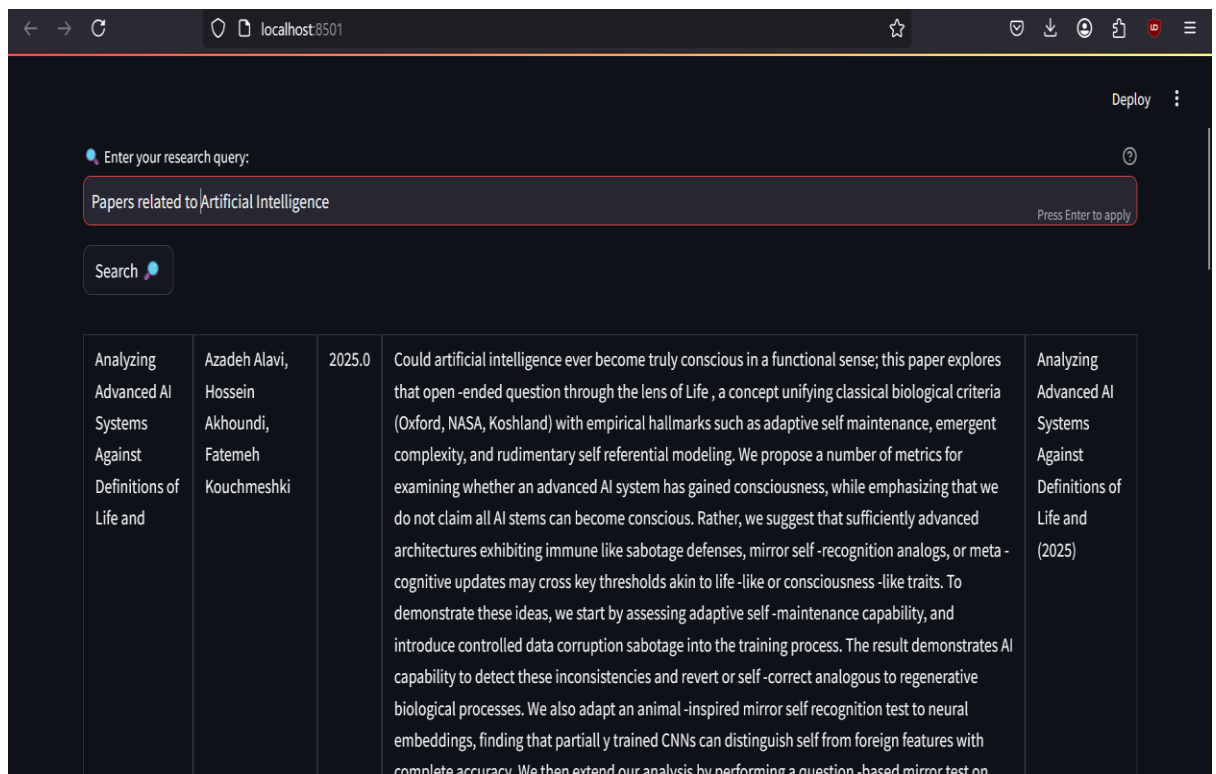


Fig.7.3 Keyword Search-1

Analyzing Advanced AI Systems Against Definitions of Life and	Azadeh Alavi, Hossein Akhoundi, Fatemeh Kouchmeshki	2025.0	Could artificial intelligence ever become truly conscious in a functional sense; this paper explores that open-ended question through the lens of Life, a concept unifying classical biological criteria (Oxford, NASA, Koshland) with empirical hallmarks such as adaptive self maintenance, emergent complexity, and rudimentary self referential modeling. We propose a number of metrics for examining whether an advanced AI system has gained consciousness, while emphasizing that we do not claim all AI stems can become conscious. Rather, we suggest that sufficiently advanced architectures exhibiting immune like sabotage defenses, mirror self-recognition analogs, or meta-cognitive updates may cross key thresholds akin to life-like or consciousness-like traits. To demonstrate these ideas, we start by assessing adaptive self-maintenance capability, and introduce controlled data corruption sabotage into the training process. The result demonstrates AI capability to detect these inconsistencies and revert or self-correct analogous to regenerative biological processes. We also adapt an animal-inspired mirror self recognition test to neural embeddings, finding that partially trained CNNs can distinguish self from foreign features with complete accuracy. We then extend our analysis by performing a question-based mirror test on five state-of-the-art chatbots (ChatGPT4, Gemini, Perplexity, Claude, and Copilot) and demonstrated their ability to recognize their own answers compared to those of the other chatbots.	Analyzing Advanced AI Systems Against Definitions of Life and (2025)
Artificial Intelligence and	Lee Peoples	2025.0	This article reports the results of a study examining the ability of legal and non-legal Large Language Models to perform legal analysis using the Issue-Rule-Application-Conclusion framework. LLMs were	No citations

Fig.7.3 Keyword Search-2

Analyzing Advanced AI Systems Against Definitions of Life and	Azadeh Alavi, Hossein Akhoundi, Fatemeh Kouchmeshki	2025.0	Could artificial intelligence ever become truly conscious in a functional sense; this paper explores that open-ended question through the lens of Life, a concept unifying classical biological criteria (Oxford, NASA, Koshland) with empirical hallmarks such as adaptive self maintenance, emergent complexity, and rudimentary self referential modeling. We propose a number of metrics for examining whether an advanced AI system has gained consciousness, while emphasizing that we do not claim all AI stems can become conscious. Rather, we suggest that sufficiently advanced architectures exhibiting immune like sabotage defenses, mirror self-recognition analogs, or meta-cognitive updates may cross key thresholds akin to life-like or consciousness-like traits. To demonstrate these ideas, we start by assessing adaptive self-maintenance capability, and introduce controlled data corruption sabotage into the training process. The result demonstrates AI capability to detect these inconsistencies and revert or self-correct analogous to regenerative biological processes. We also adapt an animal-inspired mirror self recognition test to neural embeddings, finding that partially trained CNNs can distinguish self from foreign features with complete accuracy. We then extend our analysis by performing a question-based mirror test on five state-of-the-art chatbots (ChatGPT4, Gemini, Perplexity, Claude, and Copilot) and demonstrated their ability to recognize their own answers compared to those of the other chatbots.	Analyzing Advanced AI Systems Against Definitions of Life and (2025)
Artificial Intelligence and	Lee Peoples	2025.0	This article reports the results of a study examining the ability of legal and non-legal Large Language Models to perform legal analysis using the Issue-Rule-Application-Conclusion framework. LLMs were	No citations

Fig.7.3 Keyword Search-3

CHAPTER 8

CONCLUSION

8.1 Conclusion

The implementation of a Hybrid RAG (Retrieval-Augmented Generation) system for automated literature review has significantly enhanced the efficiency and accuracy of academic research retrieval and summarization. By integrating traditional keyword-based search with semantic retrieval mechanisms using vector databases, this approach ensures a more precise and contextually relevant selection of research papers. Additionally, the incorporation of graph-based relationships between research papers further refines the retrieval process by identifying key connections between studies, leading to a more holistic understanding of a given research topic.

Through Pinecone for vector storage and retrieval, Graph Databases for relationship mapping, and LLMs (such as OpenAI's GPT or Gemini AI) for summary generation, the system successfully streamlines the literature survey process. The ability to extract and present research papers in a structured tabular format, including essential metadata like authors, publication year, objectives, and summary, has provided researchers with a powerful tool to accelerate the initial stages of academic research.

This approach mitigates the challenges associated with traditional literature review methods, such as information overload and redundancy, by automating document processing, ranking relevance, and synthesizing meaningful insights. The implementation of Hybrid RAG thus serves as an effective means of bridging the gap between raw data extraction and knowledge synthesis, making it a valuable contribution to the domain of automated literature review and scientific discovery.

8.2 Future Scope

Despite its advantages, there are several areas where this Hybrid RAG-based literature review system can be further improved and expanded. Future enhancements include expanding the system to support multi-modal data such as PDFs, images, graphs, and tables, along with integrating OCR (Optical Character Recognition) techniques for extracting text from visual elements. Real-time updates and live data extraction from

sources like Google Scholar, ArXiv, IEEE Xplore, and PubMed can be implemented to ensure an up-to-date literature review process, utilizing APIs of digital libraries for dynamic paper retrieval. Personalized research recommendations can be developed using collaborative filtering and content-based filtering algorithms to suggest relevant papers based on a researcher's history and interests. Additionally, integration with reference management tools such as Zotero, EndNote, and Mendeley would enable seamless organization and citation management, along with exporting summaries in BibTeX, JSON, and PDF formats.

Enhancing summarization and insight extraction through fine-tuned transformer models, zero-shot and few-shot learning techniques, and ontology-based retrieval methods will improve contextual understanding and keyword relevance. Advanced search capabilities with natural language understanding (NLU) can enable more interactive and multi-lingual queries. The incorporation of domain-specific knowledge graphs will further enhance relevance in niche research areas. AI-driven plagiarism detection and novelty scoring mechanisms can be integrated to verify originality and highlight emerging research trends. Additionally, deploying the system as a cloud-based service or API would increase accessibility and enable seamless integration with other platforms. Ethical considerations, such as addressing biases in LLM-generated summaries and retrieval processes, along with explainability mechanisms for transparency, are also crucial areas of development. By implementing these advancements, the Hybrid RAG-based literature review system can evolve into a more intelligent, accessible, and researcher-friendly tool, paving the way for next-generation academic research automation.

CHAPTER 9

REFERENCES

- [1] Peng, B., Zhu, Y., Liu, Y., Bo, X., Shi, H., Hong, C., ... & Tang, S. (2024). Graph retrieval-augmented generation: A survey. *arXiv preprint arXiv:2408.08921*.
- [2] Zhao, S., Yang, Y., Wang, Z., He, Z., Qiu, L. K., & Qiu, L. (2024). Retrieval augmented generation (rag) and beyond: A comprehensive survey on how to make your llms use external data more wisely. *arXiv preprint arXiv:2409.14924*.
- [3] Salemi, A., & Zamani, H. (2024, July). Evaluating retrieval quality in retrieval-augmented generation. In *Proceedings of the 47th International ACM SIGIR Conference on Research and Development in Information Retrieval* (pp. 2395-2400).
- [4] Soudani, H., Kanoulas, E., & Hasibi, F. (2024, December). Fine tuning vs. retrieval augmented generation for less popular knowledge. In *Proceedings of the 2024 Annual International ACM SIGIR Conference on Research and Development in Information Retrieval in the Asia Pacific Region* (pp. 12-22).
- [5] Fan, W., Ding, Y., Ning, L., Wang, S., Li, H., Yin, D., ... & Li, Q. (2024, August). A survey on rag meeting llms: Towards retrieval-augmented large language models. In *Proceedings of the 30th ACM SIGKDD Conference on Knowledge Discovery and Data Mining* (pp. 6491-6501).
- [6] Zhao, P., Zhang, H., Yu, Q., Wang, Z., Geng, Y., Fu, F., ... & Cui, B. Retrieval-augmented generation for ai-generated content: A survey. *arXiv 2024. arXiv preprint arXiv:2402.19473*.
- [7] Zhu, Y., Yuan, H., Wang, S., Liu, J., Liu, W., Deng, C., ... & Wen, J. R. (2023). Large language models for information retrieval: A survey. *arXiv preprint arXiv:2308.07107*.
- [8] Han, B., Susnjak, T., & Mathrani, A. (2024). Automating Systematic Literature Reviews with Retrieval-Augmented Generation: A Comprehensive Overview. *Applied Sciences*, 14(19), 9103.
- [9] Zhao, S., Yang, Y., Wang, Z., He, Z., Qiu, L. K., & Qiu, L. (2024). Retrieval augmented generation (rag) and beyond: A comprehensive survey on how to make your llms use external data more wisely. *arXiv preprint arXiv:2409.14924*.
- [10] Wang, Y., Guo, Q., Yao, W., Zhang, H., Zhang, X., Wu, Z., ... & Zhang, Y. (2024). AutoSurvey: Large Language Models Can Automatically Write Surveys. *arXiv preprint arXiv:2406.10252*.
- [11] Bründl, P., Scheck, A., Nguyen, H. G., & Franke, J. (2024). Towards a circular economy for electrical products: A systematic literature review and research agenda for automated recycling. *Robotics and Computer-Integrated Manufacturing*, 87, 102693.

- [12] Legate, A., Nimon, K., & Noblin, A. (2024). (Semi) automated approaches to data extraction for systematic reviews and meta-analyses in social sciences: A living review. *F1000Research*, 13, 664.
- [13] de la Torre-López, J., Ramírez, A., & Romero, J. R. (2023). Artificial intelligence to automate the systematic review of scientific literature. *Computing*, 105(10), 2171-2194.
- [14] Wang, S., Fan, W., Feng, Y., Ma, X., Wang, S., & Yin, D. (2025). Knowledge Graph Retrieval-Augmented Generation for LLM-based Recommendation. *arXiv preprint arXiv:2501.02226*.
- [15] Li, M., Miao, S., & Li, P. (2024). Simple is effective: The roles of graphs and large language models in knowledge-graph-based retrieval-augmented generation. *arXiv preprint arXiv:2410.20724*.